Next-Generation Intelligent Portfolio Management

by

Zijie Zhao

B.S. Nanjing University (2018)
M.S. Harvard University (2020)

Submitted to the Department of Electrical Engineering and Computer Science
in partial fulfillment of the requirements for the degree of

DOCTOR OF PHILOSOPHY

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

May 2024

Authored by:	Zijie Zhao Department of Electrical Engineering and Computer Science May 17, 2024
Certified by:	Roy E. Welsch Professor of Management, Thesis Supervisor
Accepted by:	Leslie A. Kolodziejski Professor of Electrical Engineering and Computer Science Chair, Department Committee on Graduate Students

Next-Generation Intelligent Portfolio Management

by

Zijie Zhao

Submitted to the Department of Electrical Engineering and Computer Science
on May 17, 2024 in partial fulfillment of the requirements for the degree of

DOCTOR OF PHILOSOPHY

ABSTRACT

In the fast-paced world of financial technology, the integration of advanced Natural Language Processing (NLP) and Deep Reinforcement Learning (DRL) is transforming portfolio management. This thesis presents a pioneering portfolio management framework that leverages Transformer-based models and Large Language Models (LLMs) to enhance return predictions and sentiment extraction from extensive financial texts coupled with robust DRL trading agents to optimize portfolio performance.

We introduce an adaptive retrieval-augmented framework for LLMs, finely tuned through instruction tuning to align with human instructions and incorporate market feedback. This approach enables dynamic weight adjustments within the Retrieval-Augmented Generation (RAG) module, showcasing the synergy between extracting more accurate underlying sentiment and better capturing stock movements, resulting in more profitable and robust portfolios. Additionally, we address the challenges of applying DRL to stock trading by developing the Hierarchical Reinforced Trader (HRT). This innovative strategy employs a bi-level DRL framework that combines strategic stock selection via a High-Level Controller with effective trade executions managed by a Low-Level Controller. Our results demonstrate significant enhancements in portfolio management, achieving higher Sharpe ratios than the S&P 500 benchmark in bullish markets, while also substantially reducing losses and drawdowns in bearish and volatile market scenarios. Moreover, model interpretability is crucial given the black-box nature of both LLMs and DRL models. Practitioners without a strong machine learning background require clear interpretations of model outputs. To address this, one idea is to consider features univariately, omitting feature interactions to maintain interpretability. The Univariate Flagging Algorithm (UFA) identifies optimal cut points for each feature, flags them, and summarizes them to lower dimensions for each sample. We further enhance the UFA framework within the Generalized Additive Model (GAM), extending it to a broader framework capable of modeling any data generated by exponential family distributions. Our comparative analysis on various public benchmark datasets demonstrates that our extended framework not only achieves better predictive results than the original UFA but also retains its robustness against missing and imbalanced datasets.

In conclusion, this thesis underscores the significant potential of integrating advanced NLP and DRL techniques into portfolio management, setting a new standard for intelligent financial decision-making.

Thesis supervisor: Roy E. Welsch
Title: Professor of Management

Acknowledgments

This journey has been an odyssey of growth, challenge, and discovery, and I am deeply grateful for the support and guidance I have received along the way. First and foremost, I extend my sincere appreciation to the EECS Department for providing a solid and rigorous doctoral training program. The generous funding enabled me to fully concentrate on my research, and the department's unwavering commitment to excellence set the stage for my academic endeavors. I am also profoundly thankful to the MIT Sloan School of Management, where I took many courses, fulfilled my Ph.D. minor requirement, and obtained the Business Analytics Certificate. My Ph.D. supervisor from Sloan has been an integral part of my journey, and for that, I am immensely grateful.

My heartfelt thanks go to Prof. Roy Welsch, my Ph.D. supervisor, whose influence has been pivotal during my graduate studies. Our relationship began in his 15.062 Data Mining Course when I cross-registered from Harvard. Following this, I had the privilege of working on several independent projects under his supervision. His strong recommendation letters, along with those from other supervisors I had worked with, played a crucial role in my acceptance into the Ph.D. program at MIT EECS. I have greatly appreciated the open and insightful working style with Prof. Welsch. His business acumen and solid background in statistics have been a constant source of inspiration. Our regular weekly meetings ensured steady progress in my projects, and his encouragement and valuable insights were particularly motivating when I faced challenges in my research. Notably, even during his sabbatical leave when he injured his arm and required surgery, he continued to guide my research, which deeply moved me and demonstrated his unwavering commitment.

Switching my major from Biostatistics to Computer Science was a daunting task, compounded by studying remotely from China for the first two years due to the travel restrictions of COVID-19. The initial years, primarily focused on graduate-level coursework, were challenging. I still remember the tough times when I first took graduate-level advanced systems and algorithm courses. However, looking back now, I feel that those difficult times significantly enriched my theoretical knowledge. I am deeply grateful to the senior Ph.D. students who offered their support when I first entered the program. They helped me adapt to the pace of life as a doctoral student and taught me how to balance my studies and life. Defining my research areas was a significant step, and Prof. Andrew Lo was an important advisor who guided me into the field of quantitative finance. He helped me build a clear research mindset, working with real financial data and conducting rigorous research. His mentorship provided a solid foundation for my research trajectory.

I am incredibly fortunate that Prof. Yoon Kim and Prof. Peter Szolovits agreed to join my committee during the final year of my Ph.D. Their kind guidance and suggestions

were instrumental in refining the details of my thesis. Prof. Szolovits, who also served as my Research Qualifying Examination (RQE) chair, provided invaluable feedback and future work directions. I am so grateful for his kind support.

Beyond my professors, many others have significantly contributed to my Ph.D. journey. I extend my heartfelt thanks to Prof. Leslie Kolodziejski and the EECS Graduate Office staff for their unwavering support. Their guidance and oversight were pivotal in enabling me to complete my Ph.D. in four years. I am also deeply grateful to my friends at MIT, including Deifei Zhang, Jiejin Jin, and Zhongqiang Hu, as well as those not mentioned by name but equally important. Their companionship made my life at MIT enjoyable, providing support during stressful times and sharing in my joy during moments of success.

Finally, I want to extend my heartfelt gratitude to my family for their strong encouragement. Despite having no experience in research, they supported my Ph.D. journey in every possible way. My girlfriend, also a Ph.D. student, provided immense emotional support, and we shared many significant moments together, such as internships, job searches, and eventually, my defense. Her understanding and empathy were invaluable during the most challenging times. A special note goes to my grandfather, who passed away two years ago. He was a great soldier, worker, and an outstanding entrepreneur. He raised me since I was young, and without him, I wouldn't have achieved what I have today. If he were still with us, I believe he would be proud of this moment.

Overall, I am profoundly thankful to everyone who helped me reach this significant milestone. Your support has made this journey possible and meaningful. As I move forward, I am excited about the next chapter as a quantitative researcher, where I will continue my research in quantitative finance and face the real, dynamic, and challenging financial market. My Ph.D. journey has made all of this possible.

Contents

List of Figures

List of Tables

Chapter 1

Introduction to Intelligent Portfolio Management

1.1 Background

1.1.1 Definition of Portfolio Management

Portfolio management refers to the strategic practice of constructing and managing a collection of financial assets, such as stocks, bonds, or derivatives, to align with long-term financial objectives and risk tolerance. The primary goal of portfolio management is to maximize returns while minimizing risks across the entire portfolio. A well-constructed portfolio balances these factors to achieve optimal performance.

One of the most influential theories in portfolio management is the Modern Portfolio Theory (MPT) [1], introduced by Harry Markowitz, who was awarded the Nobel Prize in 1990 for his contributions to the field. MPT provides a framework for assembling a portfolio of assets such that the overall risk (variance) is minimized for a given level of expected return, or equivalently, the return is maximized for a given level of risk. A key concept in MPT is the efficient frontier, which is graphically represented as a hyperbolic curve, as shown in **Figure 1.1**. The efficient frontier illustrates the set of optimal portfolios that offer the highest expected return for a defined level of risk or the lowest risk for a given level of expected return. Portfolios that lie below the efficient frontier are considered suboptimal because they do not provide sufficient return for the level of risk assumed. Conversely, portfolios on the upper part of the curve are efficient, achieving the best possible trade-off between risk and return. Among the portfolios on the efficient frontier, the minimum variance portfolio holds special significance. It represents the portfolio with the lowest possible risk (variance) for the expected return. This portfolio is critical in financial decision-making as it provides a baseline for comparing other portfolios.

To mathematically determine the most efficient portfolio, MPT employs quadratic programming. The objective is to minimize the portfolio variance, σ_p^2, subject to a set of linear constraints that define the lower bound for the expected return and ensure that the sum of the asset weights equals one, with all weights being non-negative. The mathematical formulation can be expressed as follows:

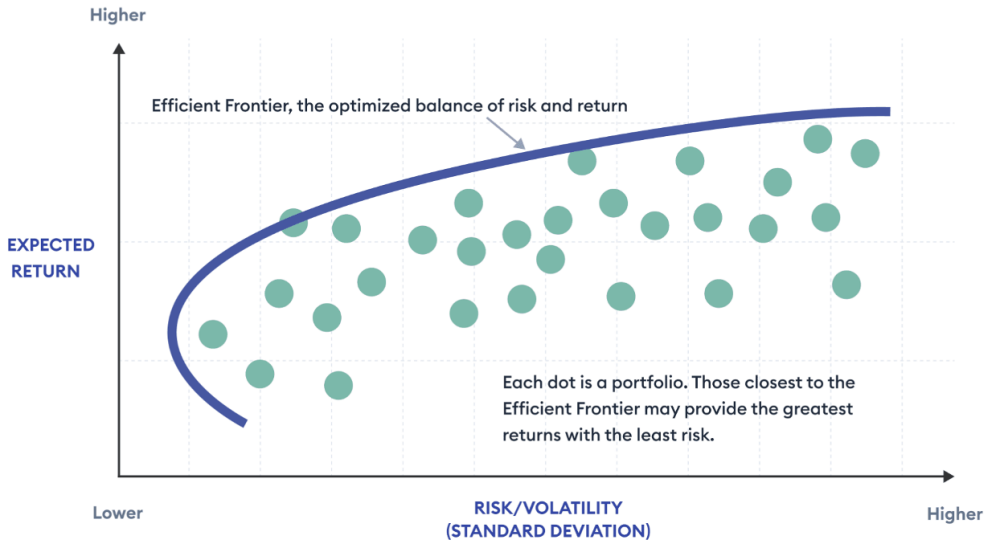


Figure 1.1: Markowitz Modern Portfolio Theory (MPT) efficient frontier.[2]

$$
\begin{aligned}
&\min \sigma_p^2 = \sum_{i=1}^{n} \sum_{j=1}^{n} w_i w_j \sigma_{ij} \\
&\text{subject to} \\
&\sum_{i=1}^{n} w_i R_i = R_p \\
&\sum_{i=1}^{n} w_i = 1 \\
&w_i \geq 0 \quad \forall i
\end{aligned}
\tag{1.1}
$$

where σ_p^2 is the portfolio variance, w_i and w_j are the weights of the assets in the portfolio, σ_{ij} is the covariance between the returns of asset i and asset j, R_i is the expected return of asset i, and R_p is the desired portfolio return.

By solving this optimization problem, investors can identify the optimal asset allocation that minimizes risk for a given return or maximizes return for a given level of risk, thereby constructing an efficient portfolio in line with MPT principles.

Traditionally, portfolio management involves analyzing historical data to predict future returns of assets using linear or deep learning models. After this predictive step, the optimization problem is solved to determine the optimal asset positions, which are then executed in trading. The classical portfolio management framework is depicted in **Figure 1.2**.

1.1.2 Next-Generation Intelligent Portfolio Management

The advent of artificial intelligence has significantly transformed asset management, enhancing various aspects such as decision-making in trading, the discovery of innovative trading

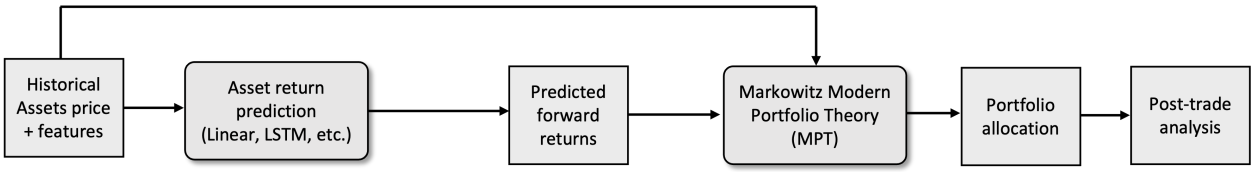


Figure 1.2: Classical portfolio management framework.

strategies, capital allocation, and risk management [3]. These advancements are paving the way for the development of next-generation, scalable, and data-driven intelligent asset management frameworks.

One key area benefiting from AI advancements is asset return prediction. Asset return, derived from price information and predicted based on historical data by identifying patterns, can be formulated as a time series forecasting problem. Among the most groundbreaking innovations in Natural Language Processing (NLP) over recent years is the Transformer model [4]. Demonstrating robust capabilities in modeling long-range dependencies and interactions in sequential data, Transformers have emerged as attractive tools for time-series modeling. Several studies affirm that Transformers outperform traditional models like ARIMA and non-attention sequence models like LSTM [5] [6] [7]. The Transformer architecture, as depicted in **Figure 1.3**, consists of an encoder-decoder structure. Both the encoder and the decoder are composed of multiple identical layers, depending on the model's size and complexity.

Each layer in the encoder comprises two main components. The first is the multi-head self-attention mechanism, which allows the model to focus on different parts of the input sequence simultaneously, capturing various aspects of the input at different levels of abstraction. Mathematically, the self-attention mechanism can be represented as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1.2}$$

where Q is the query matrix, K is the key matrix, V is the value matrix, and d_k is the dimensionality of the keys. The second component is the feed-forward neural network, which processes the output of the self-attention mechanism through a series of linear transformations and non-linear activations to produce the final output for each layer.

The decoder shares a similar structure with the encoder but includes an additional mechanism to incorporate the encoder's output. The decoder uses masked multi-head self-attention, a variant of the self-attention mechanism that prevents the decoder from attending to future tokens in the sequence, ensuring that predictions for a position depend only on known outputs. Furthermore, the decoder includes an encoder-decoder attention mechanism, allowing it to attend to the encoder's output and combine information from both the input sequence and the partially generated output sequence. Like the encoder, the decoder also employs a feed-forward neural network to process the output of the attention mechanisms.

The application of NLP-based financial forecasting techniques in building intelligent asset management has been discussed in [8]. Another significant advancement in improving the classical portfolio management framework is leveraging Large Language Models (LLMs) to mine useful signals from massive alternative data sources. At their core, language models are probabilistic models that assign a probability $P[w_1, w_2, \ldots, w_n]$ to every finite sequence

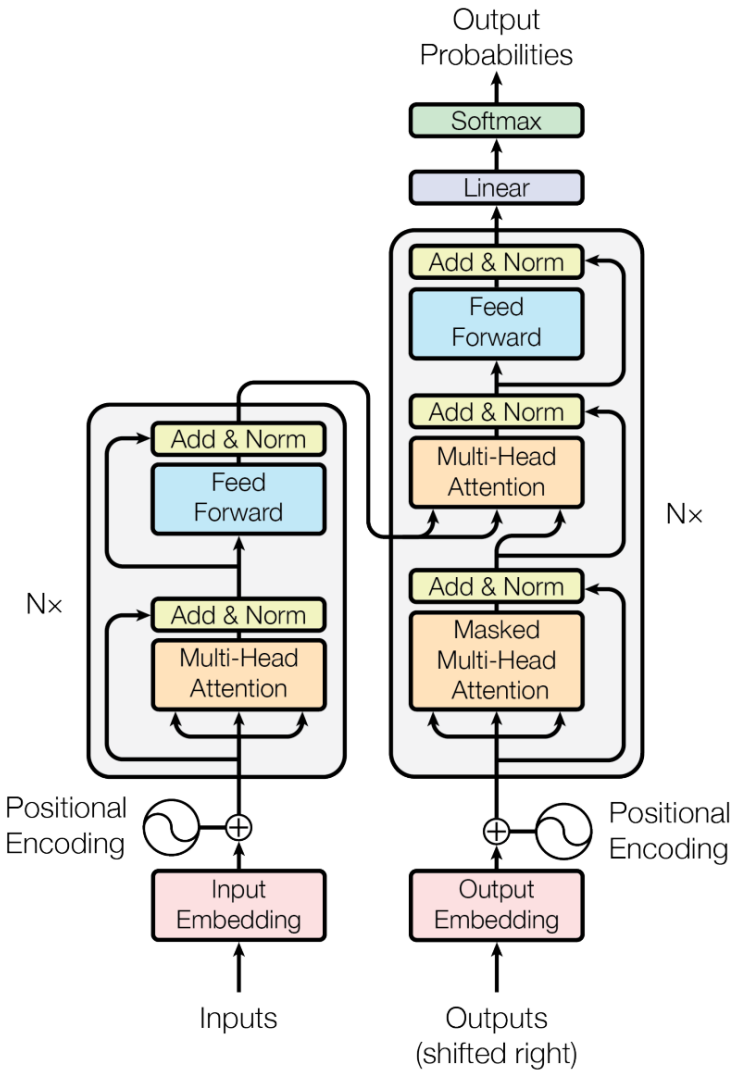


Figure 1.3: Transformer architecture. [4]

of words $w_1, \ldots, w_n$ regardless of grammatical correctness. For example, the probability of the sentence "the cat sat on the mat" can be thought of as the multiplication of a series of conditional probabilities, representing the probability of the next words given the previous segments. Mathematically, this can be expressed as:

$$\begin{aligned} P(\text{the cat sat on the mat}) = P(\text{the}) * P(\text{cat} \mid \text{the}) * P(\text{sat} \mid \text{the cat}) \\ * P(\text{on} \mid \text{the cat sat}) * P(\text{the} \mid \text{the cat sat on}) \\ * P(\text{mat} \mid \text{the cat sat on the}) \end{aligned} \tag{1.3}$$

LLMs operate on this principle but scale it up significantly. They predict the next word in a sequence by considering the preceding segment of the text. While this might seem like a simple task, achieving high accuracy requires models with billions of parameters trained on datasets containing trillions of tokens. The complexity and scale of these models enable them to perform exceptionally well across various NLP tasks. There are different types of

LLMs, each with unique characteristics and applications.

- **Open-Source LLMs**: Meta Research has released the LLaMA 2 series [9], which includes models with 7 billion (7B), 13 billion (13B), and 70 billion (70B) parameters. These models have shown impressive results across various NLP tasks. Notably, the latest series LLaMA 3[1] was released only two weeks ago, highlighting the rapid advancements in this field.

- **Closed-Source LLMs**: BloombergGPT [10], with 50 billion (50B) parameters, is a specialized model designed for the finance domain. Despite being closed-source and trained on proprietary Bloomberg data, it also achieves commendable performance in general NLP tasks.

- **Leading AI Chatbots Offering User Interface and APIs**: OpenAI's ChatGPT[2], a landmark AI conversational product released in November 2022, provides two LLMs: GPT-3.5 and GPT-4. These models have revolutionized human-computer interaction by offering sophisticated conversational abilities. Similarly, Anthropic has developed the Claude 3 series[3], including Claude 3 Haiku, Claude 3 Sonnet, and Claude 3 Opus. Google has introduced the Gemini models[4], which claim improvements over GPT-4 in specific tasks such as mathematics and creative content generation. The Gemini series includes Gemini Nano, Gemini Pro, and Gemini Ultra.

In summary, LLMs represent a significant leap in NLP capabilities, driven by their immense scale and sophisticated architectures. These models have demonstrated considerable potential in a broad spectrum of NLP tasks, ranging from natural language understanding (NLU) to generation tasks, even hinting at pathways towards Artificial General Intelligence (AGI) [11]. LLMs are primarily built upon the Transformer architecture discussed above. The versatility of the Transformer architecture has led to the development of various models, each tailored for specific tasks by employing different combinations of encoders and decoders.

- **Encoder-Only Models:** BERT (Bidirectional Encoder Representations from Transformers) [12] is a prominent example of an encoder-only model. BERT leverages bidirectional attention to capture context from both directions, making it highly effective for understanding complex language patterns. RoBERTa [13], an optimized version of BERT, enhances performance by training on larger datasets with longer sequences and dynamic masking.

- **Decoder-Only Models:** The most well-known example in this category is the GPT (Generative Pre-trained Transformer) series, including GPT-3 and GPT-4, which utilize a decoder-only architecture. These models stack multiple layers of Transformer decoders to generate coherent and contextually relevant text. ChatGPT, an extension of the GPT series, has further refined these capabilities to excel in conversational AI applications.

[1] https://ai.meta.com/blog/meta-llama-3/
[2] https://openai.com/chatgpt/
[3] https://www.anthropic.com/news/claude-3-family
[4] https://deepmind.google/technologies/gemini/

- **Encoder-Decoder Models:** Models such as BART (Bidirectional and Auto-Regressive Transformers) [14] employ both encoder and decoder components. This hybrid approach combines the strengths of bidirectional encoding and autoregressive decoding, making BART suitable for tasks like text generation and summarization.

The third potential area for improvement is portfolio allocation. While Modern Portfolio Theory (MPT) has been highly influential in the field of finance, it has several notable limitations. MPT relies on linear assumptions and efficient market hypotheses that often fail to capture the complexities of real-world markets. Additionally, it typically ignores alternative data sources that could provide deeper insights into market behavior. The theory primarily considers volatility as the measure of risk, neglecting other potential risk factors. Furthermore, MPT models are not well-suited to adapt to real-time market changes and lack end-to-end learning, as the approach requires solving the optimization problem in discrete steps rather than providing a seamless process.

Deep Reinforcement Learning (DRL), another rapidly evolving domain, has showcased impressive results in tasks such as game-playing and robotics control [15]. With its ability to learn through interaction with an unknown environment, DRL offers two key benefits for portfolio management: scalability and independence from market models. DRL presents a modern alternative to MPT for portfolio management by considering a multitude of complex financial factors as states. DRL agents can construct multi-factor models and generate algorithmic trading strategies, a task often challenging for human traders [16], [17]. Unlike MPT, DRL can process input data directly and help traders develop end-to-end trading strategies with a high degree of automation. This method eliminates many intermediate stages required in conventional approaches, offering a more adaptable and comprehensive solution. As illustrated in **Figure 1.4**, DRL in finance integrates data pre-processing, signal generation, portfolio optimization, trade execution, and post-trade analysis into a unified framework. This integration allows for real-time adaptability and continuous learning, addressing many of the limitations inherent in MPT.

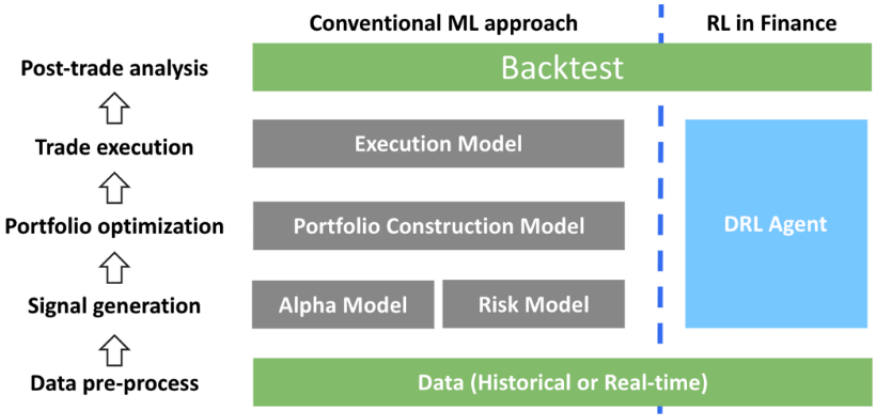


Figure 1.4: Comparison between conventional machine learning approach and RLOps in finance for an algorithmic trading process. [18]

Having discussed three potential improvements over classical portfolio management, we propose the following next-generation portfolio management framework, as shown in **Figure**

1.5. In summary, the Transformer model demonstrates better performance than linear models in capturing temporal correlations and predicting the next day's asset returns. LLMs can also mine useful signals from alternative data sources, such as sentiment analysis, which describes the current overall market preference and can be cross-validated with forward returns predicted by historical trends. For downstream portfolio allocation and trading, DRL techniques can deliver superior portfolios. We will discuss recent relevant work in these three areas in detail in the next section.

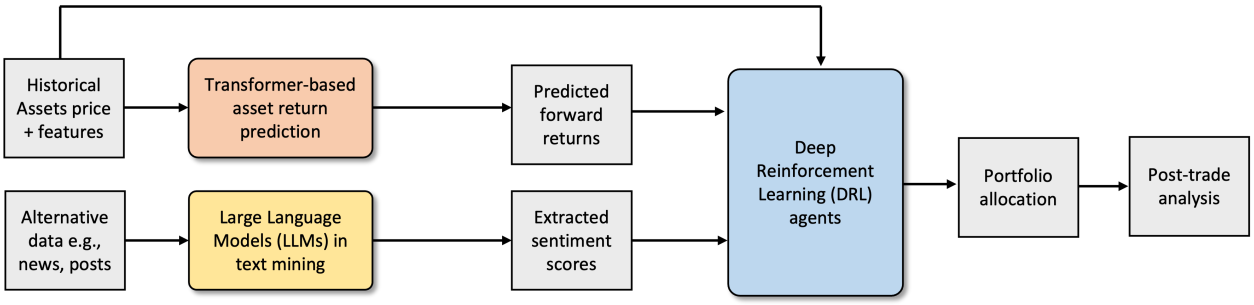


Figure 1.5: Our proposed next-generation portfolio management framework.

1.2 Related Work

1.2.1 Transformers in Asset Return Prediction

Recent years have witnessed a surge in the development of new Transformer variants for time series forecasting tasks. These variants can be broadly categorized into two groups: module-level and architecture-level. The module-level variants, which constitute a significant portion of recent work, introduce various time series inductive biases to design new modules. This group is further divided into three types: the development of new attention modules, innovative approaches for time series data normalization, and the introduction of bias for token inputs, as depicted in **Figure 1.6** [19].

The first category of module-level Transformer variants focuses on designing new attention modules, which are the most common in this field. LogTrans [5] introduces convolutional self-attention and a Logsparse mask, reducing complexity from $O(N^2)$ to $O(N \log N)$. Informer [7] achieves similar computational improvements by selecting dominant queries and designing a generative-style decoder for long-term forecasting. Pyraformer [20] uses a hierarchical pyramidal attention module to efficiently capture various temporal dependencies. FEDformer [21] performs attention operations in the frequency domain, achieving linear complexity via fixed-size frequency subsets. These models employ sparsity inductive bias or low-rank approximation to filter noise and achieve lower computational complexity.

The second category of module-level variants relates to the normalization of time series data. NSTransformer [22], for instance, addresses the overstationarization problem by modifying the normalization mechanism, thereby boosting the performance of various attention blocks.

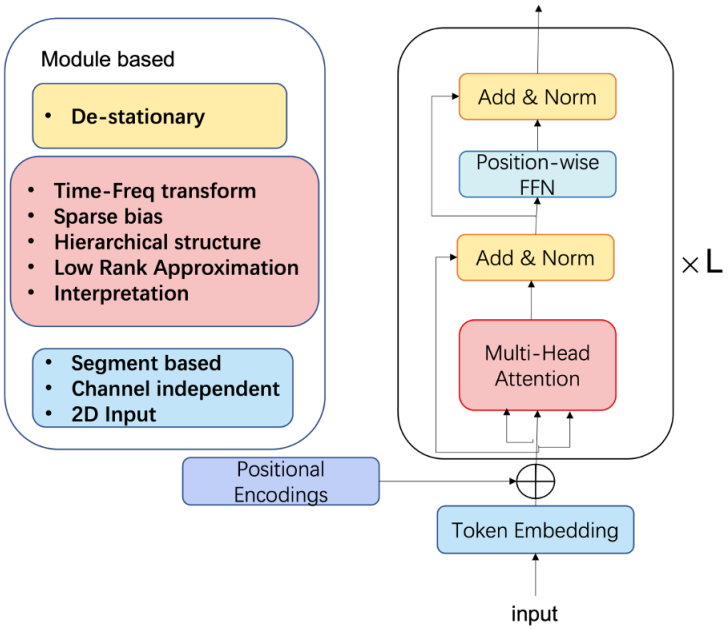


Figure 1.6: Categorization of module-level Transformer variants for time series forecasting. [19]

The third category involves utilizing bias for token inputs. Autoformer [6], for example, employs a segmentation-based representation mechanism and a simple seasonal-trend decomposition architecture. Its auto-correlation block measures time-delay similarity and aggregates top-k similar sub-series to produce output with reduced complexity.

Architecture-level Transformer variants go beyond the scope of the vanilla Transformer [4], introducing novel architectures for improved efficiency and performance. Triformer [23] designs a triangular, variable-specific patch attention that maintains a lightweight and linear complexity through its tree-type structure and variable-specific parameters.

However, some research questions the necessity of Transformers for long-term time series forecasting. For instance, DLinear [24] suggests a simpler Multilayer Perceptron (MLP)-based model can outperform some Transformer baselines in empirical studies. Similarly, TiDE [25] employs an MLP-based encoder-decoder model with a Time-series Dense Encoder for long-term time-series forecasting, demonstrating the advantages of linear models in simplicity, speed, and handling non-linear dependencies. Yet, a recent Transformer model, PatchTST [26], outperforms DLinear for long-term time series forecasting. Furthermore, model performance can greatly depend on feature construction and experimental settings. Therefore, we still focus on Transformer-based models in stock return prediction in our thesis.

In recent years, a plethora of Transformer models have been introduced and evaluated across a variety of general benchmark datasets. These datasets typically encompass fields other than financial time series data, including examples like ETT (Electricity Transformer Temperature)[5] and ECL (Electricity Consuming Load)[6]. Nonetheless, financial data presents its own set of unique challenges due to its low signal-to-noise ratio, which can compound the

[5] https://github.com/zhouhaoyi/ETDataset
[6] https://archive.ics.uci.edu/ml/datasets/ElectricityLoadDiagrams20112014

difficulty of making accurate predictions. We can apply these state-of-the-art Transformer-based models in return prediction by constructing a 3D tensor with dimensions (batch size, sequence length, features). This tensor is then input into each model to predict the return for each stock on each trading date.

1.2.2 Large Language Models (LLMs) in Financial Text Mining

Large Language Models (LLMs) have proven their efficacy across diverse Natural Language Processing (NLP) tasks. Recently, their potential has been explored within the asset management domain. BloombergGPT [10], a finance-oriented LLM with 50 billion parameters, demonstrated superior performance on financial NLP tasks without sacrificing its capability on general LLM benchmarks. BERT [12] inspired the creation of domain-specific BERT variants for the financial sector [27], [28], [29]. For example, FinBERT [29], pretrained on an extensive financial communication corpus, achieves top-tier results across multiple financial NLP tasks, including sentiment analysis, ESG classification, and forward-looking statement (FLS) classification. By utilizing FinBERT, asset managers can scrutinize alternative data sources, such as news articles and financial reports, thereby extracting invaluable insights like sentiment scores. These scores can subsequently be deployed to shape asset allocation strategies, particularly in response to sudden and unexpected price fluctuations. Similarly, RoBERTa [13] can be fine-tuned on financial sentiment datasets to leverage its improvements on the model side.

1.2.3 Deep Reinforcement Learning (DRL) in Portfolio Allocation

Recent applications of Deep Reinforcement Learning (DRL) in portfolio allocation have considered both discrete and continuous state and action spaces, employing either a critic-only, actor-only, or actor-critic approach. The critic-only learning approach, which is the most prevalent, resolves discrete action space problems using techniques such as Deep Q-learning (DQN) [30], [31] and its variants like Double DQN [32] and Dueling DQN [33]. It trains an agent on a single stock or asset [34]. However, the critic-only approach is limited as it only works with discrete and finite state and action spaces, rendering it impractical for large stock portfolios with continuous prices. The actor-only approach, utilized in [35], [36], directly learns the optimal policy. Instead of a neural network learning the Q-value, the policy is learned. Recurrent reinforcement learning is introduced to circumvent the curse of dimensionality and enhance trading efficiency [36]. The actor-only approach is competent in handling continuous action space environments. The actor-critic approach has recently been applied to automated stock trading [17], [16]. This approach involves the simultaneous updating of the actor network, representing the policy, and the critic network, representing the value function. The critic's role is to estimate the value function, while the actor updates the policy probability distribution, guided by the critic's feedback via policy gradients. The actor-critic approach has demonstrated its ability to learn from and adapt to complex and sizable environments, proving its utility in managing large stock portfolios.

To further enhance the performance of DRL in stock trading and portfolio allocation, researchers have explored various innovative approaches. [17] utilized an ensemble technique with Proximal Policy Optimization (PPO) [37], Advantage Actor Critic (A2C) [38], and

Deep Deterministic Policy Gradient (DDPG) [30] in automated stock trading. Another study [16] explored the potential application of Twin Delayed Deep Deterministic Policy Gradient (TD3) in managing a stock portfolio. Alternative research directions have sought to optimize the portfolio Sharpe ratio [39], thereby circumventing the need for forecasting expected returns. Another approach by [40] involves integrating sentiment analysis and knowledge graphs with DRL, allowing the model to incorporate external information and make more informed trading decisions. Furthermore, [41] presents a hierarchical reinforcement learning framework that integrates pair selection and trading into a unified process, demonstrating improved trading performance in different stock markets.

Our thesis is organized as follows: **Chapters 2** and **3** will discuss two works that address three major improvements to current asset management frameworks. **Chapter 4** will explore model interpretability, which was not covered in the previous chapters, and will present improvements to the Univariate Flagging Algorithm (UFA), a univariate algorithm known for its ease of interpretation. Finally, **Chapter 5** will revisit our proposed next-generation intelligent portfolio management framework, drawing conclusions and suggesting future research directions.

Chapter 2

Aligning LLMs with Human Instructions and Stock Market Feedback in Financial Sentiment Analysis

Financial sentiment analysis is crucial for trading and investment decision-making. This chapter introduces an adaptive retrieval augmented framework for Large Language Models (LLMs) that aligns with human instructions through Instruction Tuning and incorporates market feedback to dynamically adjust weights across various knowledge sources within the Retrieval-Augmented Generation (RAG) module. Building upon foundational models like LLaMA 2, we fine-tune a series of LLMs ranging from 7B to 70B in size, enriched with Instruction Tuning and RAG, and further optimized through direct feedback and Reinforcement Learning (RL)-based refinement methods applied to the source weights of RAG. Through extensive evaluation, we demonstrate that the sentiment outputs from our LLMs more accurately mirror the intrinsic sentiment of textual data, showcasing a 1% to 6% boost in accuracy and F1 score over existing state-of-the-art models and leading conversational AI systems. Moreover, the sentiments extracted are more indicative of the directions in stock price movements. On top of that, we successfully construct portfolios that yield a 3.61% higher Sharpe ratio compared to the S&P 500 baseline in bullish markets. These portfolios also demonstrate resilience in bearish markets, with a 5x reduction in return losses compared to those typically experienced by the S&P 500.

2.1 Introduction

Financial sentiment analysis is essential in interpreting investor sentiment derived from various sources such as financial articles, news, and social media. This analysis plays a pivotal role in economic forecasting, corporate finance decisions, and particularly in understanding and predicting market trends. By leveraging sentiment outputs—categorized as positive, negative, or neutral—researchers and practitioners can formulate effective trading strategies and optimize portfolio allocation. Recent advancements in Large Language Models (LLMs) have demonstrated their efficacy across numerous Natural Language Processing (NLP) tasks, including sentiment analysis. These models benefit from extensive pretraining on diverse tex-

tual corpora, allowing them to adeptly extract sentiments from financial data, as depicted in **Figure 2.1**.

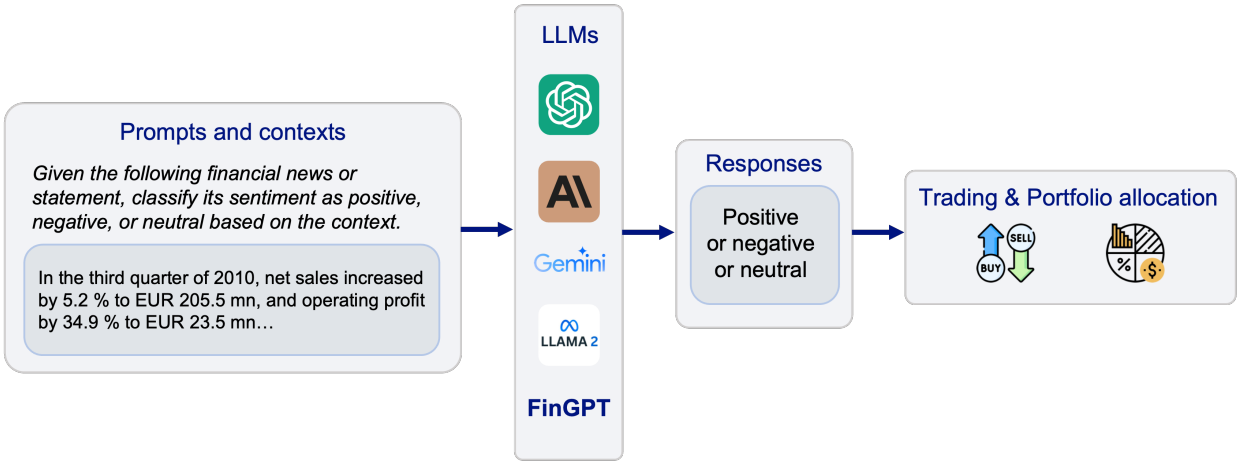


Figure 2.1: Workflow of financial sentiment analysis using LLMs.

Despite their potential, LLMs face challenges when applied to financial sentiment analysis. One significant hurdle is the disconnect between the models' pretraining objectives and the specific requirements of this field. This issue can be mitigated through targeted instruction tuning, which refines model capabilities to enhance financial sentiment prediction [42]. Additionally, the inherent brevity of sources such as news flashes and tweets often restricts the available contextual information. To address this, Retrieval-Augmented Generation (RAG) techniques have been introduced, enriching LLMs with external information during inference to improve accuracy and reliability [43].

Furthermore, recent research by Zhang et al. [44] introduced a retrieval-augmented framework for financial sentiment analysis that utilizes multi-source queries and keyword similarity-based retrieval. This approach enhances model predictions by leveraging diverse and qualitatively varied information sources. However, reliance on token similarity metrics can introduce biases, potentially favoring contexts that are misleading and fabricated. This underscores the need for careful weighting of different knowledge sources within the RAG module.

The primary objective of financial sentiment analysis is the development of trading strategies and the guidance of portfolio allocation, as illustrated in **Figure 2.1**. Therefore, integrating market feedback—specifically, aligning sentiment predictions with subsequent market returns—demonstrates a potential synergy between extracting underlying sentiment and predicting stock movements. Reflecting on this market feedback, we refine the weighting of knowledge sources in our financial LLMs' RAG module through direct adjustment or a reinforcement learning (RL)-based approach. This weight adaptation aims to optimize the contributions from various knowledge sources within the RAG module, thereby improving the overall efficacy of financial sentiment analysis. Meanwhile, using more accurate sentiment outputs as trading signals can, in turn, help predict stock movements.

Moreover, prevailing research has largely focused on LLMs of a smaller scale, approximately 7 billion parameters. The concept of emergent abilities [45] describes how increasing

model sizes unveil new capabilities in areas such as arithmetic, language comprehension, and semantic analysis. Larger models, benefiting from extensive training on broader datasets, are anticipated to offer superior contextual understanding. This potential for improved performance in tasks such as sentence classification, in our case, sentiment analysis, invites further exploration into the effects of scaling model size. This chapter addresses key research questions (RQs) vital for advancing financial sentiment analysis:

- **RQ 1**: How can we enhance the LLMs' RAG module with adaptive and non-uniform weighting of multiple knowledge sources and update the weights based on real-world market feedback?
- **RQ 2**: What is the impact of increasing model size on the performance of LLMs in financial sentiment analysis?
- **RQ 3**: Does aligning LLMs with market feedback result in sentiment predictions that more accurately reflect near-future stock price movements?

To address these RQs, we explore direct weight refinement and RL methods to fine-tune knowledge source inputs in our LLMs framework, guided by market feedback. Our extensive evaluation, encompassing a range of benchmarks and comparisons with various model sizes and cutting-edge LLMs, reveals that our advanced financial LLMs markedly outshine existing solutions in financial sentiment analysis. Our primary contributions include:

- The development of an innovative retrieval-augmented LLM framework that dynamically modulates weights in response to market feedback. This framework introduces a suite of financial LLMs finetuned on foundational models like LLaMA 2 [9], extending from pretrained to complex models enriched with Instruction Tuning and RAG, further optimized by both direct feedback and RL techniques. A detailed exposition of these models is provided in **Section 2.3**.
- The superiority of our leading model over the most recent conversational AI systems and the results of recent research in terms of accuracy and F1 score across evaluated benchmarks. We also observe a steady, though modest, enhancement in performance as LLM sizes increase from 7B to 70B parameters. Furthermore, we display the distribution of weights among the different knowledge sources consulted by the LLMs, providing a clearer understanding of the information retrieval mechanism.
- This analysis confirms the effectiveness of our LLMs in forecasting stock price movements. By implementing long-short strategies derived from sentiment outputs in both bullish and bearish markets, our best model showcases a superior Sharpe ratio when compared to the S&P 500 portfolio.

This chapter is organized as follows: **Section 2.2** reviews the background and related work. **Section 2.3** describes how to finetune financial LLMs and methods for refining knowledge source weights. **Section 2.4** details our evaluation metrics, experimental setups, and findings.

2.2 Related Work

2.2.1 Methods in Financial Sentiment Analysis

Sentiment analysis is a critical aspect of financial NLP, aiding in the interpretation of market sentiments to inform investment strategies. Initial studies employed deep learning techniques, LSTM, to handle sequential data and understand financial narratives effectively [46]. The advent of BERT [12] revolutionized NLP by providing a context-aware pretrained model. Adaptations of BERT for finance, such as FinBERT [27]–[29], have excelled in analyzing financial sentiments by navigating the intricacies of financial language. However, these methods face challenges like insensitivity to numbers, difficulty in discerning sentiment without clear context, and dealing with financial jargon and multilingual content in the face of limited labeled data. The RoBERTa model [13] is an extension and modification of the original BERT model. Due to its extended training on larger datasets and dynamic masking, RoBERTa is better at understanding context and nuances in language.

Recently, LLMs have emerged as a potent solution, offering advanced in-context learning and reasoning capabilities. They facilitate zero-shot learning, performing tasks without domain-specific training [47], [48]. BloombergGPT [10] represents a significant stride in this direction, tailor-made for financial applications. Yet, its proprietary nature and the high resource demand for training highlight the accessibility and scalability issues of such specialized models. Consequently, recent research has shifted towards fine-tuning open-source foundation models like LLaMA [9], [49] to broaden access to efficient financial LLMs.

Despite these advancements, LLMs tailored for finance still face challenges in accurate sentiment prediction due to the mismatch between their general training goals and the specific needs of financial sentiment analysis. This problem is particularly pronounced when analyzing brief content like news flashes and tweets, where limited background information impedes the LLMs' ability to correctly assess sentiments.

2.2.2 Instruction Tuning

Aligning LLMs with specific user instructions presents a notable challenge. Instruction tuning has been identified as an effective strategy for addressing this, enhancing LLMs to better respond to human feedback [42]. This approach involves training models with ***(INSTRUCTION, OUTPUT)*** pairs—where ***INSTRUCTION*** defines the task for the model and ***OUTPUT*** is the model's expected response based on the instruction [50]. Such tuning improves LLMs' performance in financial tasks by enabling them to understand and execute complex instructions more accurately. Recent research [51] adapts instruction tuning for the financial domain, demonstrating the effectiveness of tailoring LLMs to comprehend and perform tasks in investment analysis and decision-making. To make instruction tuning more efficient, Parameter Efficient Fine-Tuning (PEFT) techniques such as Low-rank Adaptation (LoRA) [52] and Quantized Low-rank Adaptation (QLoRA) [53] have been developed. These methods optimize fine-tuning by significantly reducing the number of trainable parameters, thereby making the training process more cost-effective. This innovation allows for the more practical application of LLMs in specialized fields like financial sentiment analysis without the need for extensive computing resources.

2.2.3 Retrieval Augmented Generation

Retrieval-augmented generation (RAG) merges the strengths of LLMs with the capabilities of dynamic, external databases to enhance model outputs with up-to-date, domain-specific information [43]. This method utilizes external knowledge sources as a form of non-parametric memory, seamlessly integrating retrieved data with the model's input prompt to produce enriched outputs. RAG has proven effective in complex NLP tasks, such as open-domain question answering, showcasing its ability to augment LLMs' responses with relevant context [43]. Recent advancements have tailored RAG for financial sentiment analysis, leveraging it to tackle the challenges posed by the concise and rapidly evolving nature of financial news [44]. By incorporating relevant and current information from external sources, LLMs enhanced with RAG are able to provide more accurate sentiment analysis, reducing the occurrence of hallucinations [54]. This approach is crucial for developing reliable and robust applications of LLMs in the financial domain, ensuring that sentiment analysis is reflective of the most current market conditions and insights.

2.2.4 Enhancing LLMs through Feedback

The role of feedback in the iterative improvement of LLMs has garnered increasing attention in recent research, emphasizing the models' ability to evolve and adapt through feedback integration. One approach is for LLMs to self-improve by internally generating feedback loops. The main concept involves generating an initial output with an LLM; then, the same LLM evaluates its output and uses this feedback to refine itself iteratively [55]. Beyond internal mechanisms, incorporating real-world feedback into LLMs offers a valuable approach for real-time adaptation and learning. In the case of QuantAgent [56], responses are tested in real-world scenarios to enrich the knowledge source with new insights, requiring minimal human intervention. Recent research by Lopez et al. reveals a significant relationship between market sentiments analyzed by ChatGPT and subsequent stock market behaviors, suggesting that aligning sentiments with real market returns can serve as effective feedback for guiding LLMs [57].

Furthermore, Reinforcement Learning (RL) has been recognized as a potent tool for refining the decision-making capabilities of LLMs. Inspired by Reinforcement Learning from Human Feedback (RLHF) [50], researchers have investigated various RL algorithms aimed at enhancing LLMs by learning from external feedback, thereby bolstering their reasoning processes [58]. AdaRefiner introduces a novel framework to enhance the synergy between LLMs and RL feedback, contributing to the automatic self-refinement of LLMs with RL feedback. By leveraging feedback loops, real-world interactions, and reinforcement learning strategies, LLMs can achieve greater accuracy, adaptability, and relevance across a range of applications.

2.3 Methodology

2.3.1 Overview

We present a novel, adaptive retrieval-augmented LLMs framework that integrates market feedback for more dynamic weight allocation across different knowledge sources within the RAG module. Building on the LLaMA 2 architecture [9], we introduce a suite of financial LLMs tailored for financial sentiment analysis, as depicted in **Figure 2.2**, which includes:

- **LLaMA P**: Pretrained LLaMA 2 models without any modifications.
- **LLaMA I**: Enhanced LLaMA 2 models incorporating instruction tuning for an improved understanding of human instructions.
- **LLaMA I-RAG**: LLaMA 2 models equipped with instruction tuning and a standard RAG for retrieving external information.
- **LLaMA I-RAG-DR**: LLaMA 2 models that further refine the RAG module's weights directly based on market feedback, in addition to instruction tuning.
- **LLaMA I-RAG-RL**: The most advanced iteration, combining LLaMA 2 models with instruction tuning and an adaptive RAG module, where weights are optimized through reinforcement learning techniques.

The following sections will delve into the details of each component and methodological approach.

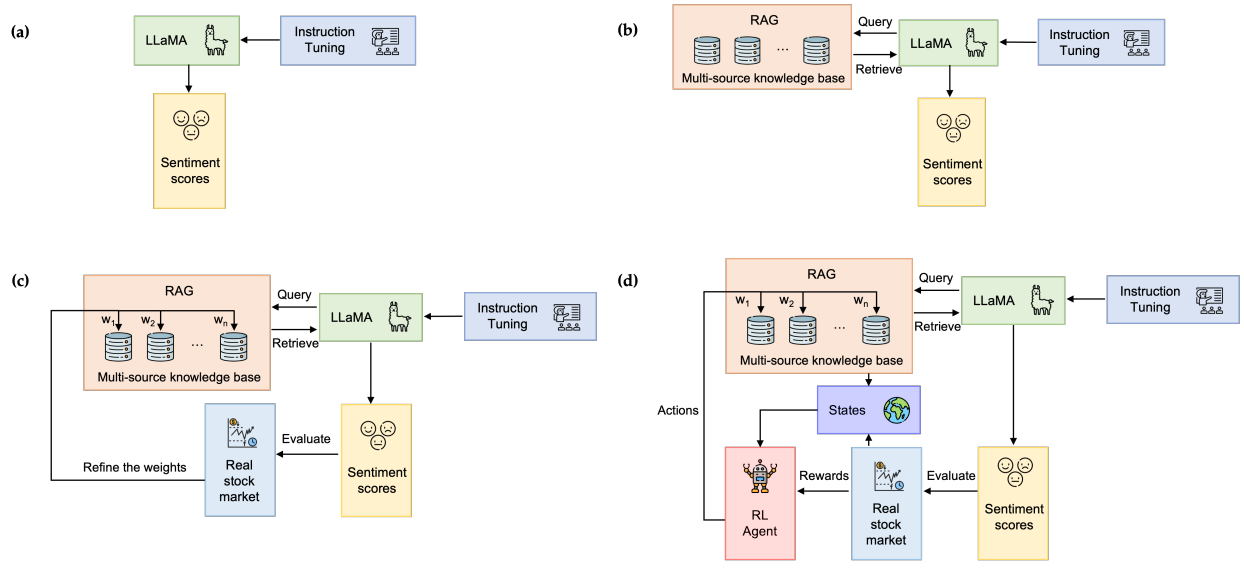


Figure 2.2: Family of financial LLMs based on the LLaMA 2. (a) LLaMA I, (b) LLaMA I-RAG, (c) LLaMA I-RAG-DF, (d) LLaMA I-RAG-RL.

2.3.2 Instruction Tuning

To implement instruction tuning for the LLaMA 2 series, we adopted a methodology analogous to the Instruct-FinGPT approach [59]. Our initial step involved constructing an instruction dataset from existing financial sentiment analysis datasets at minimal cost. We crafted ten distinct yet related instructions to elucidate the task of financial sentiment analysis, with details provided in **Appendix Section A.1**. For each data sample, we paired a randomly selected instruction with its corresponding input and output. This pairing was formatted as: "Human: ***INSTRUCTION*** + ***INPUT***, Assistant: ***OUTPUT***." Subsequently, we fine-tuned the LLMs using the causal language modeling (CLM) objective, which enhances the prediction of the next token based on prior context. Despite these efforts, the model occasionally generated extraneous free-style text that did not correspond to the targeted sentiment labels, a phenomenon observed both in the pretrained and some fine-tuned instances. In such cases, we assigned the predominant keyword to one of three predefined sentiments: positive, negative, or neutral. If this approach proved ineffective, manual verification was employed, although this was necessary in only approximately 1% of cases.

2.3.3 RAG Module

We construct an external multi-source knowledge source incorporating various sources, as detailed in **Table 2.1**. The selection criteria for these sources include the availability of automated retrieval APIs, reliability and authority, relevance to finance, data quality and format, and diversity of perspectives. These sources provide additional context for LLMs to generate sentiments. Specifically, we apply non-uniform weights $(w_1, w_2, \ldots, w_K)$ to each source and ensure these weights are normalized to sum to 1.

Table 2.1: Selected knowledge sources for RAG module.

Source	**Type**	**Primary Focus**	**Accessibility**
Bloomberg	News	Financial markets, economic news	Subscription-based
CNBC	News	Financial markets, business news	Free, with premium options
Morningstar	Research Publication	Investment research, stock ratings	Subscription-based
Reddit	Social Media	Public discussions, market sentiments	Free
Seeking Alpha	Research Publication	Company analysis, investment strategies	Free, with premium options
The Wall Street Journal	News	Business news, financial markets	Subscription-based
Twitter	Social Media	Real-time news, public sentiment	Free
Yahoo Finance	News	Stock market news, financial reports	Free, with premium options

We process original statements intended for sentiment analysis as queries by using regular expressions to extract a set of relevant keywords as queries. Then we use platform internal retrieval APIs to gather a batch of documents daily, forming a candidate pool that is pre-filtered by stock symbol and date. Then, the Weighted Overlap Coefficient (WOC) is employed to consider the weight of each source w_i and the overlap between the query and the document.

$$WOC(X, Y, w) = w \times \frac{|X \cap Y|}{\min(|X|, |Y|)} \tag{2.1}$$

where X and Y represent sets of relevant tokens [1] from the queried sentence and its context, respectively, and w is the weight corresponding to the source of Y. The top K documents, determined by WOC, are appended to the original query, along with instructions, to derive the final sentiment output. The detailed algorithm is described in **Algorithm 1**. A detailed, illustrated example of how the RAG module works is shown in **Appendix Section A.2**.

Algorithm 1 RAG with Weighted Overlap Coefficient (WOC)

Input: Query Q, Weights of knowledge source W
Output: Context C
Initialize $C \leftarrow \emptyset$
Retrieve a candidate pool of documents D, each potentially relevant to Q
Initialize an empty list P to store documents and WOC scores
for each document $d \in D$ **do**
    Obtain the weight w_j corresponding to the source of d
    Calculate $WOC(Q, d, w_j)$
    Add $(d, WOC(Q, d, w_j))$ to P
end for
Sort P in descending order based on WOC scores
Select the top K entries from P and add their documents to C
return C

2.3.4 Direct Weights Refinement by Market Feedback

In line with our discussion in **Section 2.2**, we enhance our model by comparing our sentiment predictions with actual stock returns. We calculate the daily return of a stock on day T as the percentage change in its open price from day $T-1$ to day T, indicating its stock price movement. Stock movements within ± 1 standard deviation from the mean, considered as the range for neutral movements, account for about 68% of daily price changes under a normal distribution assumption. Every day, we look at both the sentiment score and the actual stock return. We deem a prediction for day T as accurate if it meets any of the following criteria:

- The return on day $T+1$ is positive and falls outside the neutral range, matching a sentiment output "positive";
- The return on day $T+1$ is negative and falls outside the neutral range, matching a sentiment output "negative";
- The return on day $T+1$ is within the neutral range, matching a sentiment output "neutral".

If a prediction doesn't fit these scenarios, we mark it as inaccurate, which prompts a negative feedback loop. We then adjust the weights of the knowledge sources used as follows:

[1] We use the NLTK library for Named-entity recognition (NER) and remove words with neutral sentiment before performing tokenization.

For accurate predictions:

$$w_i^{\text{new}} = w_i^{\text{old}} + \alpha \tag{2.2}$$

And for inaccurate ones:

$$w_i^{\text{new}} = w_i^{\text{old}} - \alpha \tag{2.3}$$

In the formulas above, $i \in K$, we only adjust the weights for sources we actually feed into LLMs; the rest remain unchanged. The constant parameter α, which influences how much we adjust the weights, is set to 1e-4 in our experiments. We aggregate the weight changes and update weights on a batch basis. The specifics of this approach are detailed in **Algorithm 2**.

Algorithm 2 Direct Refinement of Knowledge Source Weights by Market Feedback

Input: Initial uniform weights $W = \{w_1, w_2, \ldots, w_k\}$, Source usage per feedback $S = \{s_1, s_2, \ldots, s_n\}$, Market feedback $F = \{f_1, f_2, \ldots, f_n\}$, Learning rate $\alpha > 0$, Batch size B
Output: Refined weights W'
for $b = 1$ to B **do**
  Initialize $\Delta W = \{0, 0, \ldots, 0\}$ with length k to store weight changes for each batch
  for $i = 1$ to n within batch b **do**
    for each source $j \in s_i$ **do**
      if f_i indicates a correct prediction **then**
        $\Delta w_j \leftarrow \Delta w_j + \alpha$
      else
        $\Delta w_j \leftarrow \Delta w_j - \alpha$
      end if
    end for
  end for
  for $j = 1$ to k **do**
    $w_j \leftarrow \max(0, w_j + \Delta w_j)$ to ensure non-negative
  end for
  Normalize W' such that $\sum_{j=1}^{k} w'_j = 1$
end for
return W'

2.3.5 RL-based Weights Refinement by Market Feedback

In this section, we describe how we refine the weights of knowledge sources in the RAG module using Reinforcement Learning (RL), opting for a more dynamic approach compared to direct adjustments. This method allows for a more sophisticated and potentially more effective mechanism for adapting weights based on feedback, leveraging the strengths of RL in dealing with continuous action spaces and optimizing long-term rewards.

Our RL setup features a continuous action space where actions, represented as a vector $\mathbf{w} = (w_1, w_2, \ldots, w_K)$, detail the weights for the knowledge sources. We constrain each weight w_i to fall within $[0, 1]$, ensuring the total weight set sums to 1. This approach builds on the similar market feedback mechanism outlined in **Section 2.3.4**, with a reward function that assigns +1 for correct sentiment predictions and -1 for incorrect ones. The state for our RL model captures the current condition of RAG along with relevant market information, such as current weights, historical accuracy rate, average overlap coefficient with the query across different knowledge sources, and historical stock return and technical indicators (more details are shown in **Appendix Section A.3**).

In this chapter, we adopt Proximal Policy Optimization (PPO) [37], a reinforcement learning algorithm noted for its stability, efficiency, and ease of implementation, qualities particularly valuable in the volatile environment of stock trading. PPO is a type of policy gradient method, which optimizes decision-making policies directly and is well-suited for scenarios with large or continuous action spaces.

PPO addresses a common challenge in training policy gradient methods: significant policy degradation due to large, destabilizing updates. It achieves this by introducing a novel objective function that limits the size of policy updates, promoting incremental learning and preventing large shifts that could negatively affect performance. This moderation is particularly crucial in maintaining the delicate balance between exploration of new strategies and exploitation of known profitable behaviors.

The core mechanism of PPO involves a probability ratio, $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$, which compares the likelihood of taking an action under the new policy versus the old policy. This ratio is incorporated into PPO's clipped objective function:

$$J^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}(s_t, a_t), \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}(s_t, a_t) \right) \right] \tag{2.4}$$

In this equation, $\hat{A}(s_t, a_t)$ represents the advantage function, estimating the relative benefit of choosing a particular action compared to the baseline. The clipping operation restricts the ratio $r_t(\theta)$ within the range $[1-\epsilon, 1+\epsilon]$, controlling the extent of policy updates and thus enhancing the stability of the training process. For more details, please refer to the original PPO paper [37].

Our PPO-based reinforcement learning architecture consists of several fully connected layers designed to process state inputs effectively. The architecture is divided into two primary components: a policy network and a value network. The policy network is responsible for determining the action outputs, specifically the allocation of weights across K knowledge sources within the RAG module. Instead of using a softmax activation layer typically suited for discrete action spaces, we employ a linear activation function in the final dense layer of the policy network to produce a continuous action vector. A post-processing normalization step ensures that these weights sum to one. Concurrently, the value network, branching from a shared dense layer, features a single output neuron with linear activation to estimate the state's value, facilitating an assessment of the current policy's expected long-term returns. Details of this method are fully laid out in **Algorithm 3**.

Algorithm 3 RL-based Refinement of RAG Weights by Market Feedback using PPO

Input: Initial normalized weights $W = \{w_1, w_2, \ldots, w_K\}$, State features: current weights, historical accuracy, overlap coefficients, market indicators
Output: Refined weights W'
Initialize policy network π_θ for predicting weight adjustments
Initialize value network V_ϕ for state value estimation
while not converged **do**
  for each iteration **do**
    Observe state s (including market indicators and performance metrics)
    for each environment interaction within the iteration **do**
      Predict action $a = (w_1, \ldots, w_K)$ using $\pi_\theta(s)$
      Renormalize a to ensure $\sum w_i = 1$ for action execution
      Apply a, observe new state s', and calculate reward r based on prediction against market movements
      Update state $s \leftarrow s'$
    end for
    Compute advantage estimates $\hat{A}$ based on rewards and V_ϕ predictions
    Update π_θ by maximizing PPO objective w.r.t θ
    Update V_ϕ by minimizing value function loss w.r.t ϕ
  end for
end while
Output: Optimized weights W' from refined policy π_θ

2.4 Performance Evaluation

In this section, we assess the efficacy of our proposed suite of financial LLMs for the financial sentiment analysis task. We compare the results with state-of-the-art approaches from previous studies and current cutting-edge conversational AI systems. To assess the effectiveness of each model, we report accuracy and F1 scores on benchmark datasets.

2.4.1 Datasets

To fairly compare results with existing research [44], [59], [60], we used the same public datasets that are accessible through HuggingFace for training and testing the model. Specifically, we use the following datasets to train and run inference for the LLaMA I and LLaMA I-RAG models, boosting performance by instruction tuning and RAG technique.

The training dataset is a combination of the training split of Twitter Financial News Sentiment (TFNS)[2] and the FiQA SA dataset[3], resulting in a total of 10,504 samples. This combined dataset was adapted to fit an instruction-following format suitable for instruction tuning. For model evaluation, we utilized the validation segment of the TFNS dataset alongside the Financial PhraseBank (FPB) dataset[4], totaling 7,234 samples. A notable challenge with these datasets is their frequent omission of stock symbols and specific date tags—information crucial for maximizing the RAG module's retrieval capabilities.

For the LLaMA I-RAG-DF and LLaMA I-RAG-RL models, the term "train" refers to the process of applying techniques outlined in **Section 2.3.4** and **2.3.5** to fine-tune RAG weights, rather than directly modifying the LLMs' model weights. This process necessitates the inclusion of real market feedback, demanding specific data such as timestamps and stock symbols. To meet this requirement, we manually created a specialized dataset comprising 20,000 news headlines, with each randomly selected stock and date ranging from 01/01/2018 to 12/31/2020. Each headline is annotated with relevant temporal and stock symbol information, alongside the subsequent day's stock return. Performance metrics for these models were then evaluated using the same validation sets from TFNS Val and FPB as the other models for consistency.

2.4.2 Model Training

For efficient parameter fine-tuning of LLMs, we employed Low-Rank Adaptation (LoRA) [52], setting the rank to 8 and the alpha value to 32. We specifically targeted LoRA modules such as "q_proj", "k_proj", "v_proj", "down_proj", "gate_proj", and "up_proj". Our training utilizes the AdamW optimizer, with a training epochs of 10, a batch size of 32, a starting learning rate of 5e-5, and a weight decay rate of 0.1. To ensure efficiency, particularly given the LLaMA 2 model's maximum context capacity of 4,096 tokens, we implemented a practical token limit of 2,048 for each input sample. All models undergo training using

[2] https://huggingface.co/datasets/zeroshot/twitter-financial-news-sentiment
[3] https://huggingface.co/datasets/pauri32/fiqa-2018
[4] https://huggingface.co/datasets/financial_phrasebank

Table 2.2: Comparative Performance of different LLMs. The error terms represent the standard deviation calculated across ten independent training experiments, each with a different random seed.

Model	FPB		TFNS (Val)	
	Accuracy	F1	Accuracy	F1
Our financial LLMs (70B)				
LLaMA P	0.652	0.425	0.605	0.408
LLaMA I	0.738 ± 0.018	0.718 ± 0.018	0.834 ± 0.016	0.765 ± 0.015
LLaMA I-RAG	0.781 ± 0.019	0.741 ± 0.012	0.883 ± 0.016	0.819 ± 0.010
LLaMA I-RAG-DF	0.812 ± 0.018	0.782 ± 0.022	0.905 ± 0.017	0.870 ± 0.015
LLaMA I-RAG-RL	0.810 ± 0.024	0.781 ± 0.021	0.910 ± 0.021	0.878 ± 0.019
Pre-trained language model				
RoBERTa	0.723 ± 0.021	0.689 ± 0.011	0.804 ± 0.014	0.741 ± 0.009
Closed-sourced LLMs				
BloombergGPT	-	0.511	-	-
GPT-3.5 Turbo	0.654	0.529	0.673	0.484
GPT-4 Turbo	0.773	0.702	0.835	0.794
Gemini Pro	0.725	0.673	0.809	0.775
Claude-3	0.759	0.697	0.812	0.786
Recent LLMs research				
Zhang et al. (2023) [44]	0.758	0.739	0.863	0.811
Fatemi & Hu (2023) [60]	0.807	0.780	0.903	0.878

bfloat16 mixed-precision on a robust hardware setup of eight A100 (40GB) GPUs at Lambda Labs[5], completing within a few hours.

For fine-tuning the RoBERTa model used in **Section 2.4.3**, we downloaded the pre-trained RoBERTa Large model from Hugging Face[6]. A classification head was appended to tailor it for categorizing text into sentiment classes. This model was fine-tuned on the exact same benchmark datasets using the AdamW optimizer, with a learning rate of 2e-5 and a batch size of 16, running for 4 epochs to ensure precise convergence and prevent overfitting.

In our Proximal Policy Optimization (PPO) configuration, we set a policy learning rate of 5e-4, an update frequency of 10 epochs, a discount factor of 1 for immediate reward evaluation, and a clip ratio of 0.2 to moderate policy updates. It is worth noting that, while we selected PPO for its balance of efficiency and performance, the design of our LLaMA I-RAG-RL model is compatible with a variety of reinforcement learning algorithms, not limited to PPO. In terms of potential multiple sources of randomness such as model initialization, data mini-batch shuffle during training, and random sampling of news to compute sentiment scores, etc., we conducted ten independent experiments under the same setting with respect to different random seeds.

2.4.3 Results

Model Performance Comparison

To address **RQ 1** as outlined in **Section 2.1**, we developed two methods for constructing an adaptive Retrieval-Augmented Generation (RAG) system, as detailed in **Sections 2.3.4** and **2.3.5**. To evaluate their empirical performance rigorously in financial sentiment analysis, we benchmarked our LLaMA with 70 billion parameters against widely recognized pre-trained language models, such as RoBERTa Large (335M parameters), and several leading conversational AI systems, including OpenAI's GPT-4-0125-preview [61], GPT-3.5-turbo-0125 [50], Anthropic's Claude-3-Opus [62], and Google's Gemini-Pro [63]. Additionally, we included BloombergGPT [10], a first-of-its-kind, finance-focused LLM in our benchmark, relying on its reported metrics on the Financial PhraseBank (FPB) dataset. Our analysis incorporates insights from recent research [44], [60], with detailed results presented in **Table 2.2**.

Table 2.2 illustrates the progressive performance gains achieved with increasingly sophisticated models. On the TFNS validation dataset, for instance, we observed a 37.85% improvement in accuracy when LLaMA P was enhanced with instruction tuning to create LLaMA I. The integration of the RAG module further advanced LLaMA I-RAG's accuracy by 5.88%. The modest uplift from RAG is due to the limited availability of date information and pertinent named entities in tweets and news segments, which constrains the RAG's retrieval capabilities. The LLaMA I-RAG-RL model exhibited an additional 3.06% gain in accuracy, attributed to reweighting the sources in the RAG module based on their reliability and relevance as indicated by market feedback. Thus, incorporating the feedback that optimizes source weights in the LLMs' RAG module improves sentiment extraction from financial contexts.

[5] https://lambdalabs.com/service/gpu-cloud
[6] https://huggingface.co/FacebookAI/roberta-large

On the FPB dataset, there was a notable improvement in accuracy from the transition of LLaMA P to LLaMA I-RAG-DF. It is observed that LLaMA I-RAG-RL did not show a significant improvement over LLaMA I-RAG-DF, suggesting that a simple weight refinement strategy is effective for this dataset.

It is also noteworthy that the RoBERTa model, after fine-tuning, performed only slightly inferior to LLaMA I. Among the closed-source LLMs, GPT-4 Turbo remains the standout performer across the datasets evaluated. However, it is crucial to recognize that classification is merely one facet of NLP capabilities. Before concluding GPT-4 Turbo's superiority over Gemini and Claude, it is imperative to apply benchmarks across more complex tasks such as reasoning, coding, and creative writing to ensure a comprehensive evaluation [64]. Our leading model, LLaMA I-RAG-RL, surpasses recent research proposals in both accuracy and F1 score, highlighting the effectiveness of refining the RAG module through direct adjustment or reinforcement learning (RL) methods informed by market feedback.

Influence of Model Size on Performance

Prior studies [44], [59], [60] have primarily evaluated LLMs ranging in size from 250M to 7B parameters. In pursuit of **RQ 2**, we broadened this scope by training models up to 70B parameters, adhering to the methodologies delineated in **Section 2.3.1**. As illustrated in **Figure 2.3**, instruction tuning emerged as the most substantial factor in improving F1 scores across various model sizes, consistent with earlier findings. We noted only a slight enhancement in performance when scaling the model size from 7B to 13B, and then to 70B. Interestingly, the 7B variant of the LLaMA I-RAG model attained weighted F1 scores comparable to those of the 70B LLaMA I model, despite having 10 times fewer parameters. Considering the significant increase in computational resources and time required for training larger models, our results suggest that merely expanding model size is less efficient compared to employing techniques like RAG, particularly when dealing with the succinct nature of the financial news segments, headlines, and tweets in our case.

Influence of Knowledge Source Weights in RAG

This section investigates the impact of applying the direct weights refinement and RL-based weights refinement methods on adapting the source weights within the RAG module. We assessed the final distribution of source weights utilized by our 70B optimized LLMs as follows: For both LLaMA I-RAG-DF and I-RAG-RL models, we employed similarity-based retrieval using **Equation 2.1** and selected the top K sources with the highest WOC scores. In contrast, for the LLaMA I-RAG model, we uniformly set $w = 1/K$ for all sources. We recorded the retrieval history to track which sources were accessed by each sample and subsequently calculated their proportion by sources. The outcomes are depicted in **Figure 2.4**.

Although the two refinement methods resulted in differing weight distributions, a notable pattern emerged with the RL-based refinement leading to more pronounced imbalances. Specifically, only Bloomberg and The Wall Street Journal surpassed the original uniform weight level (12.5%) after applying either refinement method, as denoted by the gray horizontal line in the figure. Common to both methods was an increase in the weights

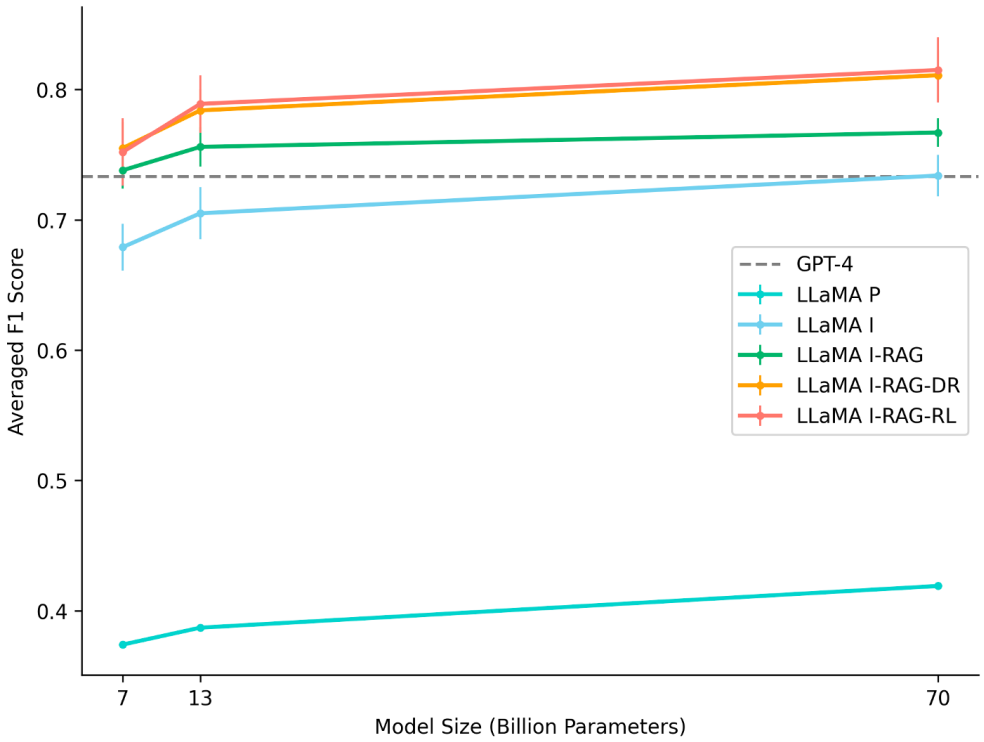


Figure 2.3: The impact of model size on weighted F1 score across test datasets. The length of the error bar extending from this central point throughout this chapter represents the standard deviation calculated across ten independent experiments, each with a different random seed. The grey horizontal line indicates the weighted F-1 score obtained by GPT-4 Turbo.

assigned to Bloomberg and The Wall Street Journal, ranging between 5% to 10%, and a reduction in the weight for socially-driven platforms like Reddit to between 2% to 3%. This adjustment aligns with the understanding that postings on Reddit generally possess lower reliability and authority compared to established news platforms. Furthermore, the diverse and broad nature of discussions on Reddit may dilute its correlation with actual stock market movements. Therefore, models adjusted their dependence on such sources based on market feedback. Nevertheless, models still allocated minor weights to sources like Reddit and Twitter, underscoring the importance of source diversity to RAG.

Portfolio Construction Based on LLMs Sentiments

To address **RQ 3**, we investigated the effectiveness of Large Language Models' (LLMs) sentiment outputs in predicting stock returns. We developed a strategy that involved constructing long-short portfolios and performing backtests. Specifically, trades were executed at the opening price at 9:30 AM on day T, with the assumption that all trades were completed at this time. Sentiment scores were derived daily by averaging the sentiment values of ten randomly selected news headlines and tweets, collected between 9:30 AM of the previous day $(T-1)$ and 9:30 AM of the current day (T). Positive, neutral, and negative sentiments

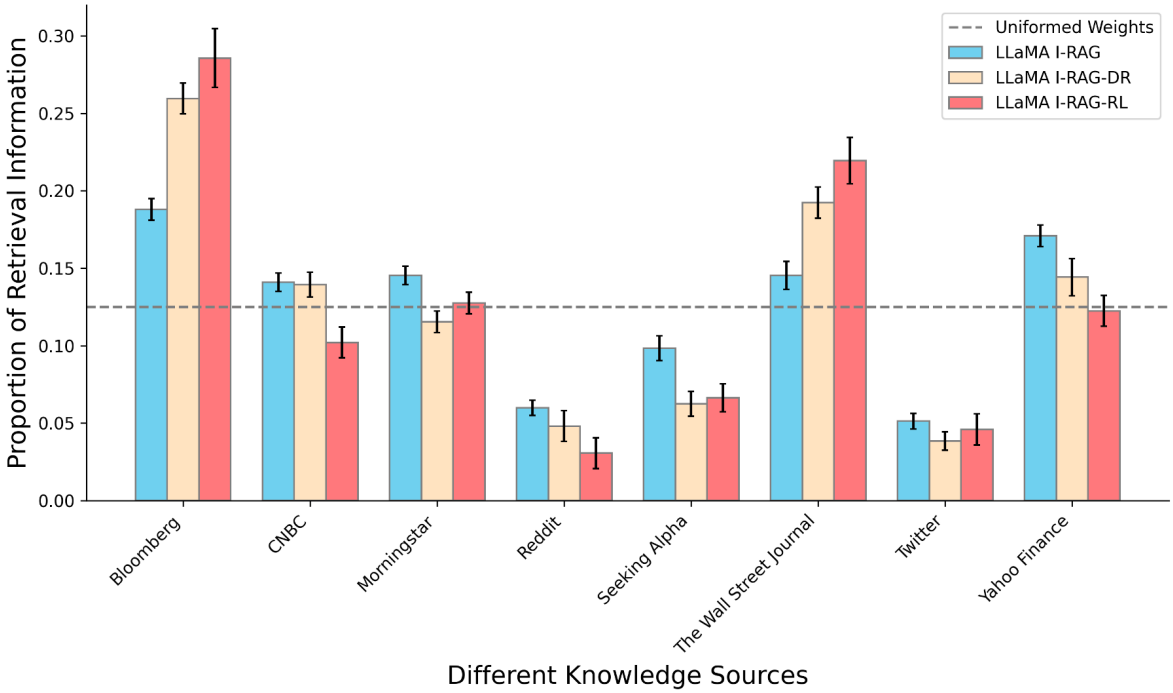


Figure 2.4: Weights distribution across different knowledge sources of RAG. The gray horizontal line represents the uniform initialization level (12.5%) for the eight evaluated knowledge sources.

were quantified as 1, 0, and -1, respectively[7].

We constructed an equal-weighted, zero-cost portfolio that longed (bought) stocks with positive sentiment scores (ranging from 0.1 to 1) and shorted (selled) those with negative sentiment scores (ranging from -1 to -0.1). Scores within the -0.1 to 0.1 range were deemed neutral, prompting us to hold positions. Our analysis was confined to constituents of the S&P 500 to ensure sufficient liquidity and used this index as a benchmark for U.S. market performance. The analysis covered two periods with different market conditions: 01/01/2021, to 12/31/2021, which provided a scenario of strong recovery and bull market trends—ideal for assessing how the strategy captures growth during economic rebound and low interest rates; and 01/01/2022, to 12/31/2022, which presented a challenging backdrop with high inflation, interest rate hikes, and bear market conditions, testing the strategy's resilience and adaptability to adverse shifts. Transaction costs were set at 10 basis points per trade to cover exchange fees, execution fees, and SEC fees. We tracked the daily cumulative returns of the portfolios constructed using our financial LLMs and compared them with the S&P 500 as a baseline, as illustrated in **Figure 2.5**. Detailed evaluation metrics are shown in **Table 2.3**.

The results, as depicted in **Figure 2.5a**, demonstrate an overall bullish market trend, evidenced by a clear upward trajectory for all LLMs and the S&P 500 baseline. Despite some volatility, the final cumulative returns at year-end show an increase with the use of

[7]For models incorporating RAG, to prevent redundant information, we retrieve information randomly without replacement.

Table 2.3: Comparative Evaluation of Performance. The error terms represent the standard deviation calculated across ten independent experiments, each with a different random seed.

(a) Year 2021 (bullish market)

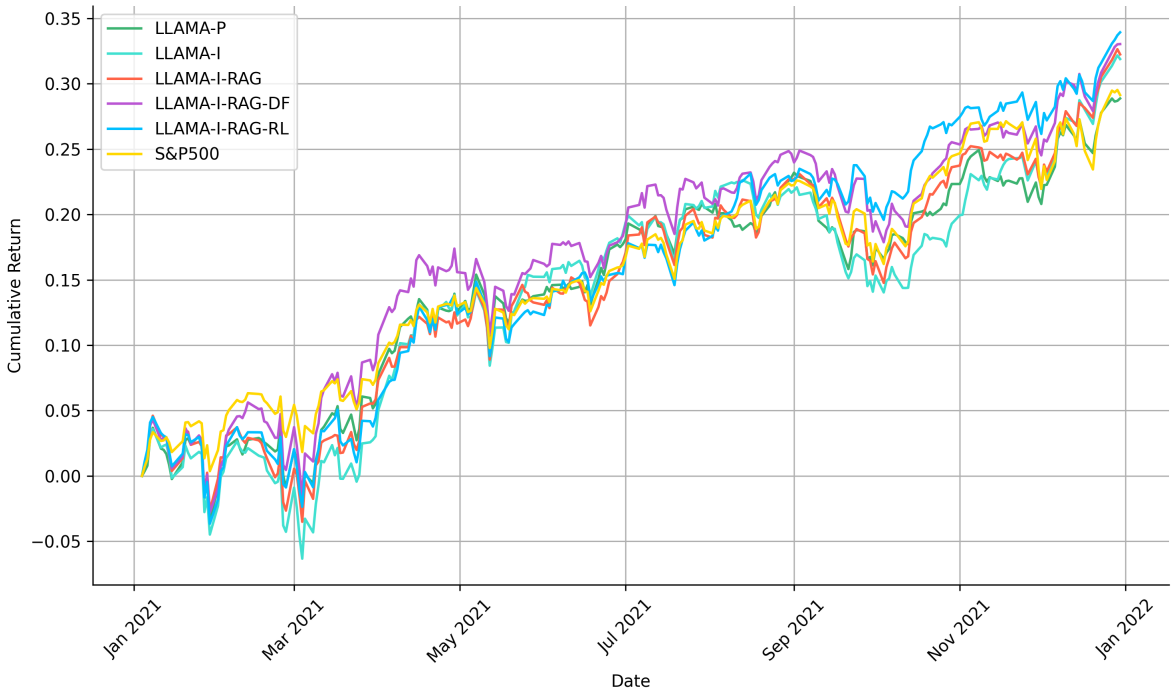
Metric	LLaMA P	LLaMA I	LLaMA I-RAG	LLaMA I-RAG-DF	LLaMA I-RAG-RL	S&P500
Cumulative Return	0.2887 ± 0.008	0.3188 ± 0.009	0.3224 ± 0.007	0.3303 ± 0.010	0.3393 ± 0.011	0.2913
Annualized Return	0.2935 ± 0.008	0.3242 ± 0.008	0.3278 ± 0.007	0.3358 ± 0.010	0.3450 ± 0.010	0.2961
Annualized Volatility	0.1468 ± 0.006	0.1608 ± 0.005	0.1528 ± 0.004	0.1555 ± 0.007	0.1464 ± 0.008	0.1303
Sharpe Ratio	1.9990 ± 0.098	2.0154 ± 0.080	2.1457 ± 0.072	2.1598 ± 0.117	2.3557 ± 0.146	2.2736
Max Drawdown	-0.0667 ± 0.012	-0.0956 ± 0.009	-0.0776 ± 0.010	-0.0683 ± 0.008	-0.0781 ± 0.009	-0.0521

(b) Year 2022 (bearish market)

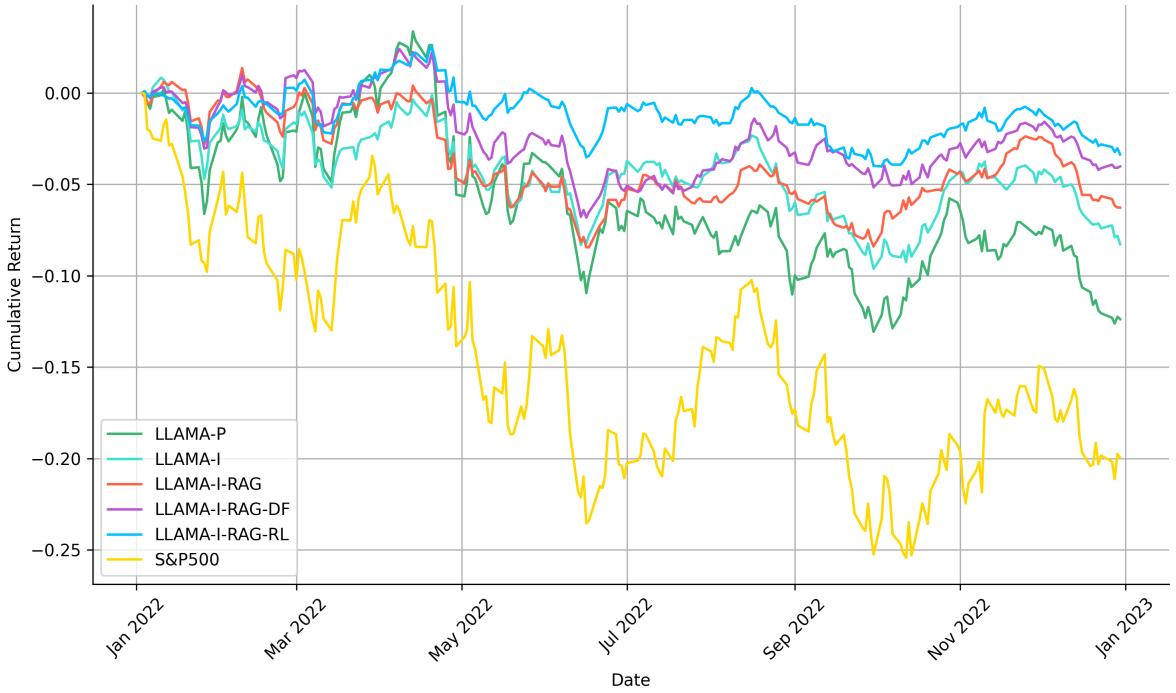
Metric	LLaMA P	LLaMA I	LLaMA I-RAG	LLaMA I-RAG-DF	LLaMA I-RAG-RL	S&P500
Cumulative Return	-0.1238 ± 0.009	-0.0828 ± 0.003	-0.0627 ± 0.005	-0.0403 ± 0.005	-0.0337 ± 0.004	-0.1995
Annualized Return	-0.1251 ± 0.009	-0.0837 ± 0.003	-0.0635 ± 0.005	-0.0407 ± 0.004	-0.0341 ± 0.005	-0.2016
Annualized Volatility	0.1471 ± 0.008	0.0990 ± 0.009	0.0788 ± 0.007	0.0780 ± 0.010	0.0608 ± 0.011	0.2416
Sharpe Ratio	-0.8507 ± 0.077	-0.8457 ± 0.083	-0.8053 ± 0.096	-0.5222 ± 0.084	-0.5614 ± 0.131	-0.8344
Max Drawdown	-0.1590 ± 0.014	-0.1037 ± 0.015	-0.0967 ± 0.012	-0.0901 ± 0.012	-0.0640 ± 0.011	-0.2543

more sophisticated LLMs. All models outperformed the baseline market portfolio, which solely invests in S&P 500 constituent stocks without leveraging sentiment analysis. Notably, only our top-performing model, LLaMA I-RAG-RL, achieved a better Sharpe ratio of 2.3557 compared to the S&P baseline at 2.2736. This outcome may be attributed to the higher volatility associated with LLM portfolios, suggesting that a more complex trading strategy, such as reinforcement learning, could be advantageous compared to our simpler strategy here that solely relies on stock sentiments.

For the year 2022, as illustrated in **Figure 2.5b**, a similar performance enhancement is observed with the adoption of more sophisticated models. Although all LLMs' portfolios registered negative cumulative returns in response to the stock market downturn and overall market decline, these portfolios still managed to mitigate losses and significantly decrease the maximum drawdown compared to the S&P 500 baseline. The LLaMA I-RAG-DF model achieved a Sharpe ratio comparable to that of the LLaMA I-RAG-DF model but with a slightly larger drawdown, underscoring the effectiveness of this simpler approach. The improved cumulative returns, under adverse market conditions, highlight the capability of our finely tuned and optimized LLMs to generate sentiment outputs that are not only more accurate but also yield more profitable trading signals.



(a) Year 2021 (bullish market)



(b) Year 2022 (bearish market)

Figure 2.5: Cumulative return curves of different investment strategies and S&P 500. Values are computed as the mean of ten independent training experiments, each with a different random seed.

Chapter 3

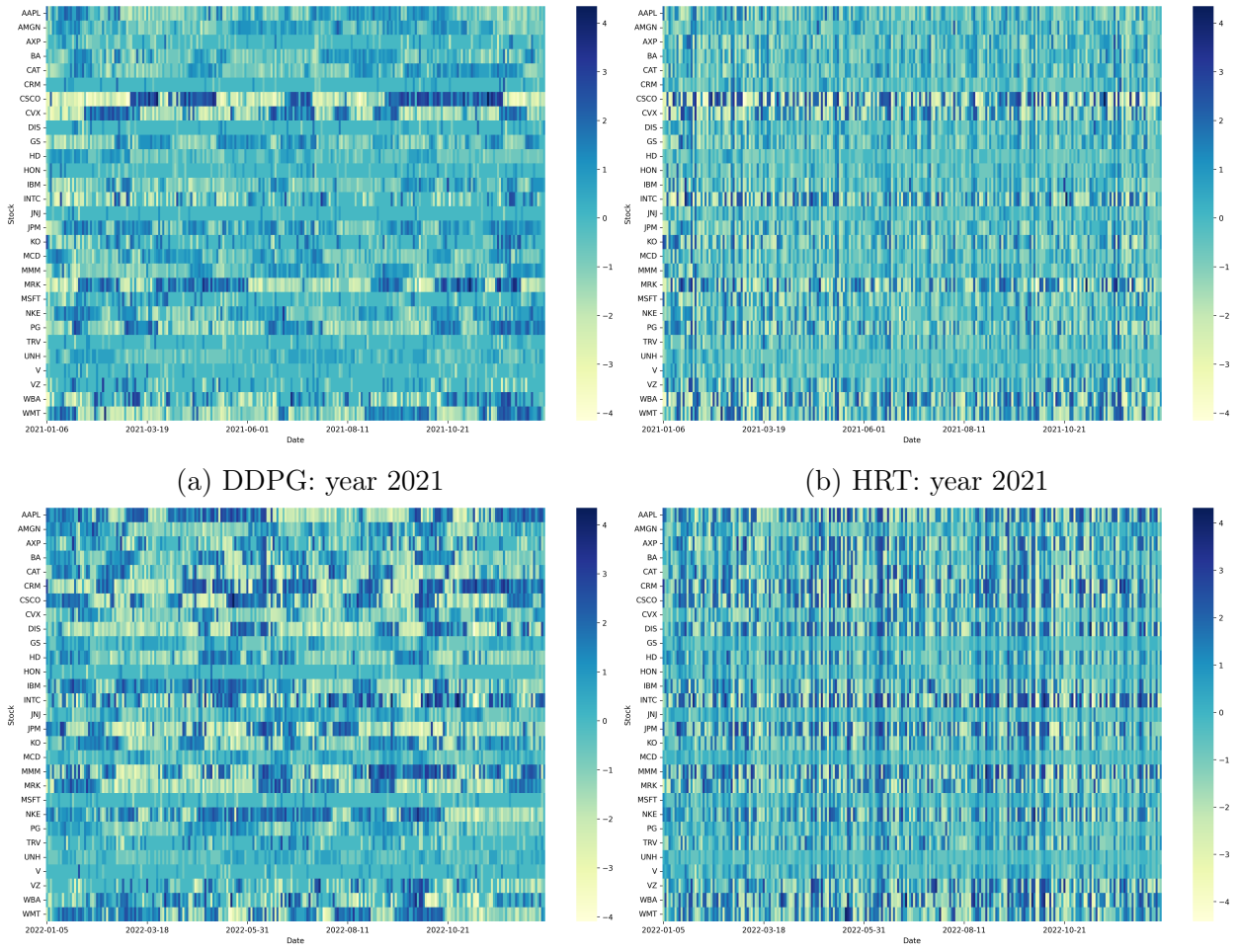
Hierarchical Reinforced Trader (HRT): A Bi-Level Approach for Optimizing Stock Selection and Execution

Leveraging Deep Reinforcement Learning (DRL) in automated stock trading has shown promising results, yet its application faces significant challenges, including the curse of dimensionality, inertia in trading actions, and insufficient portfolio diversification. Addressing these challenges, we introduce the **H**ierarchical **R**einforced **T**rader (**HRT**), a novel trading strategy employing a bi-level Hierarchical Reinforcement Learning framework in this chapter. The HRT integrates a Proximal Policy Optimization (PPO)-based High-Level Controller (HLC) for strategic stock selection with a Deep Deterministic Policy Gradient (DDPG)-based Low-Level Controller (LLC) tasked with optimizing trade executions to enhance portfolio value. In our empirical analysis, comparing the HRT agent with standalone DRL models and the S&P 500 benchmark during both bullish and bearish market conditions, we achieve a positive and higher Sharpe ratio. This advancement not only underscores the efficacy of incorporating hierarchical structures into DRL strategies but also mitigates the aforementioned challenges, paving the way for designing more profitable and robust trading algorithms in complex markets.

3.1 Introduction

Profitable automated stock trading strategies are pivotal for investment companies and hedge funds. A classical method is Harry Markowitz's Modern Portfolio Theory (MPT) [1], which determines the optimal portfolio allocation by calculating the expected returns and the covariance matrix of stock prices. This optimization aims to either maximize returns for a given risk level or minimize risk for a specified return range. However, implementing MPT can be complex, especially when portfolio managers wish to dynamically adjust decisions at each time step and consider additional factors. An alternative approach models the stock trading problem as a Markov Decision Process (MDP) [65], solved using dynamic programming. Nevertheless, this model's scalability is constrained by the expansive state spaces inherent in real stock markets.

Recent research has turned to Deep Reinforcement Learning (DRL) methods for stock trading [16], [17]. DRL overcomes scalability issues by using deep neural networks to approximate complex functions, solving MDPs without the limitations of traditional models. Liu, Xiao-Yang, et al. [66] formalize the stock trading problem as an MDP and employ Deep Deterministic Policy Gradient (DDPG) [30] to discover optimal trading strategies that yield higher cumulative returns and Sharpe ratios in the volatile stock market. Subsequent research integrates the strengths of DDPG, Proximal Policy Optimization (PPO) [37], and Advantage Actor Critic (A2C) [38] into an ensemble strategy [17], adapting robustly to varying market conditions. Despite these advancements, several challenges persist in applying DRL to stock trading:



(a) DDPG: year 2021

(b) HRT: year 2021

(c) DDPG: year 2022

(d) HRT: year 2022

Figure 3.1: Trading operations heatmap on DJIA 30 stocks for 2021 and 2022. The heatmaps depict the log values of trading operations, with a manually set trading threshold of $h_{max} = 100$. Each subfigure corresponds to a different year and trading strategy.

- **Curse of Dimensionality**: The computational complexity, sample inefficiency, and potential training instability escalate as the number of stocks increases, expanding the

dimensionality of data and the state and action spaces exponentially. For instance, if the action for a single stock is defined as $a \in \{-k, \ldots, -1, 0, 1, \ldots, k\}$, representing sell, hold, and buy actions, the action space becomes $(2 \times k + 1)^N$, where N is the number of market stocks. This complexity has limited the validation of current research to a small asset scale, ranging from Dow Jones 30 constituent stocks to only tens of assets.

- **Inertia or Momentum Effect**: DRL agents tend to repeat a previous action (buy, sell, or hold) based on the reward received, without necessarily considering the currently most profitable action. If an agent receives a large reward for a particular action (buy, sell, or hold), it may exploit this action in subsequent steps. Even though DDPG introduces action exploration through the addition of noise to the actions selected by its deterministic policy, we still observe crowded or clustered trading operations in **Figure 3.1** under the example of Dow Jones 30 constituent stocks portfolio.

- **Insufficient Diversification**: Diversification, a core principle of finance aimed at risk mitigation, is compromised when DRL agents focus repeatedly on a narrow selection of stocks. This behavior, evidenced in **Figure 3.1**, increases exposure to sector-specific risks, making the portfolio more susceptible to adverse developments within those sectors.

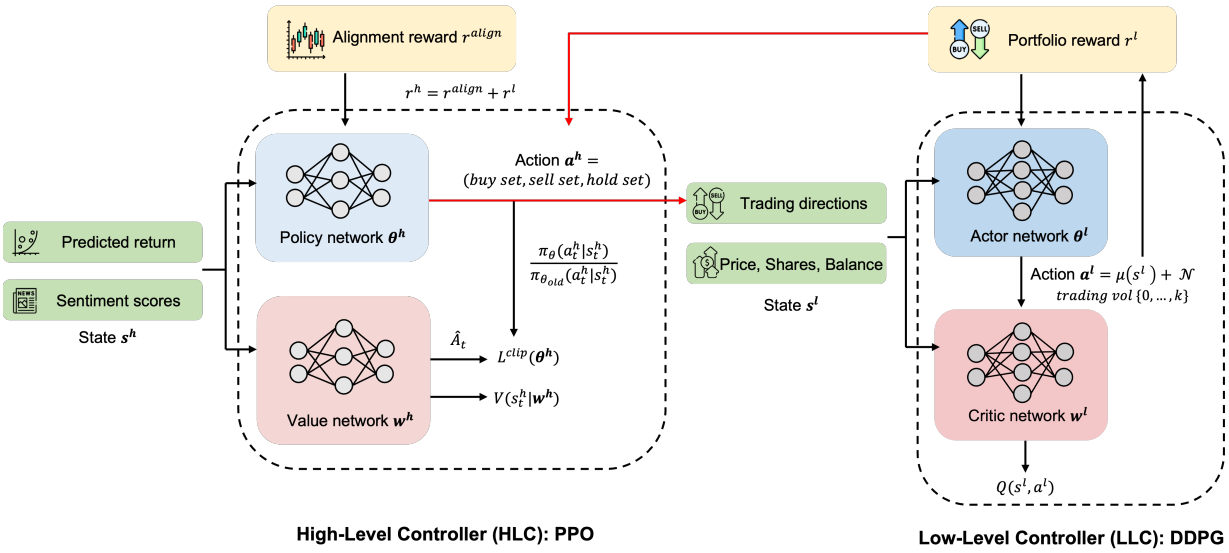


Figure 3.2: Overview of the Hierarchical Reinforced Trader (HRT) architecture. Interactions between the HLC and LLC are indicated by the red arrows.

To mitigate the three issues mentioned above and to enhance performance and deliver superior trading strategies, we introduce the **H**ierarchical **R**einforced **T**rader (**HRT**), an innovative approach to stock trading that utilizes a Hierarchical Reinforcement Learning (HRL) framework [67]. Our HRT agent is structured around two principal components, each serving distinct but complementary roles in the trading strategy: (1) **High-Level Controller (HLC)**: Positioned at the strategic apex of the hierarchy, the HLC's mandate is to determine the subset of stocks to buy, sell, or hold, effectively executing stock selection.

(2) **Low-Level Controller (LLC)**: ollowing the HLC's directives, the LLC is tasked with refining these decisions by optimizing the trade volumes for the selected stocks, thereby determining the optimal number of shares to transact. By dividing the trading strategy into high-level stock selection and low-level trade execution, the HRT agent demonstrates potential for substantial performance improvements and the ability to address traditional challenges. Our primary contributions include:

- Introducing a novel HRT agent utilizing the Hierarchical Reinforcement Learning (HRL) framework, and proposing an algorithm named Phased Alternating Training to jointly train the HLC and LLC.
- By testing and validating our results on a substantially larger stock pool, the S&P 500, we consistently demonstrated a positive and higher Sharpe ratio compared to standalone DDPG and PPO approaches across various market conditions.
- While a few studies have explored the application of Hierarchical Reinforcement Learning (HRL) in stock trading and portfolio management [41], [68], [69], to our knowledge, this work is the first to elucidate how integrating the HRL framework with DRL agents can effectively mitigate the issues previously discussed.

The chapter is organized as follows: **Section 3.2** reviews the background and related work. **Section 3.3** describes details of the HRT agent. **Section 3.4** details our data processing, evaluation metrics, experimental setups, and results.

3.2 Related Work

3.2.1 Deep Reinforcement Learning for Trading

Deep Reinforcement Learning (DRL) has significantly advanced the automation of stock trading, offering dynamic optimization through continuous market interaction, recognizing complex non-linear patterns, and integrating intermediate stages (e.g., modeling, trading signal generation, portfolio optimization, and trade execution) within a singular DRL framework [18]. Prior studies have experimented with various DRL approaches, including Proximal Policy Optimization (PPO) [37], Advantage Actor Critic (A2C) [38], and Deep Deterministic Policy Gradient (DDPG) [30], alongside its enhancement, Twin Delayed DDPG (TD3) [70]. Subsequent research has explored ensemble strategies, demonstrating superior performance over individual DRL agents [17]. Expanding DRL with additional information, such as incorporating optimistic or pessimistic reinforcement learning influenced by prediction errors, has shown promising directions [71]. Moreover, enriching state representations with more features or signals has proven beneficial for agents to grasp market dynamics more effectively. Adaptive DDPG extensions, including sentiment-aware approaches, have enhanced the model's robustness [72]. Combining sentiment analysis and knowledge graphs further refines algorithmic trading strategies [40]. A comprehensive review of these advancements is provided in [73].

3.2.2 Hierarchical Reinforcement Learning

DRL works well on a wide range of problems where the state space and action space are reasonably small. However, there are certain problems where DRL is insufficient or takes too much time to train. Hierarchical Reinforcement Learning (HRL) offers a divide-and-conquer strategy, segmenting overarching problems into more tractable subproblems. HRL decomposes decision-making into hierarchical policies: high-level policies focus on coarse-grained decisions, while low-level policies attend to fine-grained, detailed actions. This layered approach aids in navigating large action spaces more efficiently. There is limited research on HRL in trading, but some research exist. Wang, Rundong, et al. [69] have developed a hierarchical reinforced stock trading system for portfolio management, where a high-level policy allocates portfolio weights to maximize long-term profits, and a low-level policy optimizes share transactions within shorter windows to reduce trading costs. There are also HRL systems developed for specialized trading tasks, like High Frequency Trading [68], and Pair Trading [41].

3.3 Methodology

3.3.1 Overview

Our Hierarchical Reinforced Trader (HRT) agent introduces a novel approach to algorithmic trading by applying Hierarchical Reinforcement Learning (HRL). It splits the trading process into two distinct but interrelated decisions, aiming to improve trading performance through an in-depth understanding of market dynamics and execution efficiency. A schematic of our HRT framework is shown in **Figure 3.2**, outlining the agent's two main components: the High-Level Controller (HLC) and the Low-Level Controller (LLC).

High-Level Controller (HLC): The HLC plays a crucial role in analyzing market conditions and sentiments to determine the key trading directions—buy, sell, or hold—and to select stocks. Due to its strategic importance, Proximal Policy Optimization (PPO) is chosen for the HLC for its efficiency in managing discrete action spaces, simplifying the decision-making process.

Low-Level Controller (LLC): Following the HLC's strategy, the LLC hones these directions, focusing on the exact quantities of shares to trade. Deep Deterministic Policy Gradient (DDPG) is selected for the LLC. DDPG's ability to handle continuous action spaces and maintain stable learning progress makes it ideal for the detailed task of executing trades in the stock market.

Together, the HLC and LLC work towards the ultimate goal of maximizing long-term portfolio performance. The HLC's decisions inform the LLC's actions, specifying trading directions for each stock. These instructions are seamlessly incorporated into the LLC's state inputs, crucially influencing its operational strategy. The results of the LLC's trades, measured through reward signals aimed at maximizing portfolio values, are relayed back to the HLC, ensuring alignment towards a common goal. These interactions are highlighted by red arrows in **Figure 3.2**.

3.3.2 Stock Selection with High-Level Controller

The High-Level Controller (HLC) within the Hierarchical Reinforced Trader (HRT) agent is specifically designed to address the stock selection challenge. It discerns which subset of stocks to buy, sell, or hold based on strategic analysis. The components of the DRL agent include:

State space. We identify two predictive sources for the state: $s^h = [\boldsymbol{fr}, \boldsymbol{ss}]$, where $\boldsymbol{fr}$ signifies predicted forward returns from historical price and volume data, and $\boldsymbol{ss}$ captures sentiment scores derived from alternative textual data, such as news or tweets. Predictive price or forward return is one of best best predictive signals in designing trading strategies and also serve as the prediction target [74]. Predicted return and sentiment score undergo cross-validation to improve predictive accuracy.

Action space. The action executed by the HLC for stock i, noted as $a_i^h \in \{1, -1, 0\}$, maps to buying, selling, or holding, respectively. The complete action space covers 3^N, with N indicating the total number of stocks traded.

Reward. The HLC's reward, $r^h(s, a, s')$ [1], merges the real-world price movement alignment reward with the downstream feedback from the Low-Level Controller (LLC), as detailed in **Section 3.3.3**. This approach ensures HLC actions not only aim to refine immediate trading directions but also support the overarching objective of enhancing portfolio value.

The alignment reward for stock i, $r_i^h(s, a_i, s')$, correlates with action a_i and the actual price change ΔP_i. The sgn$(\cdot)$ function, returning -1 for negative inputs, +1 for positive inputs, and 0 for neutral input, is employed to determine rewards. A reward of 1 is granted if the action aligns with the real stock return, -1 if it contradicts the stock return, and no reward or penalty is applied if the action is to hold.

$$r_i^{align}(s, a_i, s') = \begin{cases} \text{sign}(a_i) \cdot \text{sgn}(\Delta P_i) & \text{if } a_i \neq 0 \\ 0 & \text{if } a_i = 0 \end{cases} \tag{3.1}$$

The comprehensive reward for the HLC, $r^h(s, a, s')$, is a linear mix of the sum of r_i^{align} for all n stocks and the LLC reward. The weighting factor α_t starts near 1, emphasizing the HLC's alignment with stock price movements, and gradually decreases to focus more on the LLC's reward, following an exponential decay $\alpha_t = \alpha_0 \cdot e^{-\lambda \cdot t}$, where we set $\alpha_0 = 1$ and $\lambda = 0.001$ in our following experiments.

$$r^h(s, a, s') = \alpha_t \sum_{i=1}^{n} r_i^{align}(s, a_i, s') + (1 - \alpha_t) r^l(s, a, s') \tag{3.2}$$

For the HLC's architecture, we select PPO, which moderates policy updates to prevent detrimental performance shifts. This process calculates the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ between new and old policies, incorporating a clipping mechanism in the objective function:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}\left(r_t(\theta), 1 - \epsilon, 1 + \epsilon\right) \hat{A}_t \right) \right] \tag{3.3}$$

[1] Variables are simplified here and subsequently; correct notation includes superscripts: a^h, s^h, r^h.

Here, $\hat{A}(s_t, a_t)$ denotes the advantage function estimate, and the clipping limits the ratio $r_t(\theta)$ to $[1-\epsilon, 1+\epsilon]$, opting for the minimum value to restrict excessive policy changes and ensure training stability. The details of the algorithm are outlined in **Algorithm 4**. After training the Proximal Policy Optimization (PPO) algorithm, we identify which subsets of stocks to buy, sell, or hold. This information is then passed to the Low-Level Controller (LLC) to determine the optimal trading volume.

Algorithm 4 PPO for High-Level Controller (HLC) in HRT Agent

Initialize policy network $\pi(a|s, \boldsymbol{\theta}^h)$ with weights $\boldsymbol{\theta}^h$
Initialize value network $V(s|\boldsymbol{w}^h)$ with weights $\boldsymbol{w}^h$
for iteration $= 1$ to M **do**
    Collect a set of trajectories $\mathcal{D} = \{s, a, r_h, s'\}$ by executing the current policy $\pi(a|s, \boldsymbol{\theta}_h)$ in the environment.
    For each trajectory in $\mathcal{D}$, compute r_h as a linear combination of the alignment reward and the LLC reward: $r_h = \alpha_t \sum_{i=1}^{n} r_i^{align} + (1-\alpha_t) r^l$, where α_t adjusts over time.
    Calculate rewards-to-go $\hat{R}_t$ and advantage estimates $\hat{A}_t$ using the collected data.
    for epoch $= 1$ to P **do**
        Update the policy by maximizing the PPO-Clip objective:

$$L(\boldsymbol{\theta}_h) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \cdot \hat{A}_t, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right) \cdot \hat{A}_t \right) \right]$$

        with $r_t(\theta) = \frac{\pi(a|s, \boldsymbol{\theta}^h)}{\pi_{\text{old}}(a|s, \boldsymbol{\theta}^h))}$ and ϵ as a hyperparameter.
        Update the value network by minimizing the squared-error loss:

$$L(\boldsymbol{w}^h) = \hat{\mathbb{E}}_t \left[\left(V(s|\boldsymbol{w}^h) - \hat{R}_t \right)^2 \right]$$

    end for
end for

3.3.3 Executing Trading with Lower-Level Controller

Given the sets that the HLC decides to buy or sell, the Lower-Level Controller (LLC) refines this decision by determining the optimal quantity of shares to trade. Since the direction is already established, the LLC can concentrate on optimizing k within the bounds set by $h_{\max}$, streamlining the training process. Additionally, for stocks designated to be held, there's no need to engage the LLC in determining the optimal k. For simplicity and consistency with current research, we adopt a setting similar to that described by Liu, Xiao-Yang, et al. [66].

State space. The state space, represented as $s^l = \left[\boldsymbol{p_t}, \boldsymbol{h_t}, b_t, \boldsymbol{a^h}\right]$, includes information on stock prices $\boldsymbol{p_t}$, stock holdings $\boldsymbol{h_t}$, and the remaining cash balance b_t. Furthermore, as depicted in **Figure 3.2**, we augment $\boldsymbol{a^h}$ with decisions from the HLC, specifying which stocks are selected for buying, selling, or holding.

Action space. The action space for a single stock is defined as $\{0, 1, \ldots, k\}$, where $k > 0$ represents the number of shares to be traded, subject to the maximum limit $k \leq h_{\max}$.

This action space is normalized to [-1, 1] because using a continuous action space facilitates the modeling of transactions across multiple stocks, avoiding the computational challenges posed by a large discrete action space.

Reward. The reward function, $r^l(s, a, s')$ [2], quantifies the change in portfolio value from trades executed at time t as reflected in stock price updates at $t + 1$. The portfolio value is computed as $\boldsymbol{p}^T\boldsymbol{h} + b$, with an acknowledgment that maintaining positions may affect the portfolio's value due to price changes.

The LLC employs the Deep Deterministic Policy Gradient (DDPG) framework for implementation. At each timestep, the LLC executes an action a in state s, earns a reward r, and transitions to a new state s'. These transactions (s, a, s', r) are stored in the replay buffer R. A batch of N transitions is drawn from R to update the Q-value y_i as follows:

$$y_i = r_i + \gamma Q'\left(s_{i+1}, \mu'\left(s_{i+1}|\boldsymbol{\theta}^{l'}\right)\right), \quad i = 1, \cdots, N \tag{3.4}$$

Next, the critic network is updated by minimizing the loss function $L(\boldsymbol{w}^l)$, which calculates the expected difference between the target critic network Q' and the actual critic network Q:

$$L(\boldsymbol{w}^l) = \mathbb{E}_{s,a,r,s'\sim\text{buffer}}\left[\left(y_i - Q(s, a|\boldsymbol{w}^l)\right)^2\right]. \tag{3.5}$$

The detailed algorithm of the LLC is illustrated in **Algorithm 5**. After training DDPG, we obtain a continuous vector that describes the number of shares to be traded. This vector is then scaled back and discretized by rounding to the nearest integer to determine the actions to buy, sell, or hold. Following these actions, trades are executed to generate portfolios.

3.3.4 Joint Training of HLC and LLC

To train our HRT agent, we propose a phased alternating training method (**Algorithm 6**). This approach initiates with the optimization of the HLC to establish a strategic foundation for trading decisions. Subsequently, we focus on training the LLC, while freezing the HLC's parameters. An iterative alternating process then refines the strategies of both the HLC and LLC, ensuring that strategic and execution tactics are consistently aligned and enhanced through mutual feedback. This feedback loop between the HLC and LLC fosters complementary learning, whereby the strategy adjustments of each controller are informed by insights from its counterpart, thereby boosting overall efficacy.

The initial focus on HLC training addresses the critical importance of accurate trading directions. Incorrect directions could undermine portfolio performance, regardless of the optimization level achieved by the LLC in determining trade volumes. A proficiently trained HLC sets the stage for effective trade execution managed by the LLC. Furthermore, dynamically integrating the rewards received by the LLC into the HLC's training process—achieved through a linear combination with exponential decay in α_t—progressively harmonizes strategic decisions with execution capabilities, cultivating a unified operational framework.

[2]Variables are simplified here and subsequently; accurate notation includes superscripts: a^l, s^l, r^l.

Algorithm 5 DDPG for Lower-Level Controller (LLC) in HRT Agent

Randomly initialize critic network $Q(s, a|\boldsymbol{w}^l)$ and actor $\mu(s|\boldsymbol{\theta}^l)$ with weights $\boldsymbol{w}^l$ and $\boldsymbol{\theta}^l$
Initialize target networks Q' and μ' with weights $\boldsymbol{w}^{l'} \leftarrow \boldsymbol{w}^l$, $\boldsymbol{\theta}^{l'} \leftarrow \boldsymbol{\theta}^l$
Initialize replay buffer R
for episode = 1 to M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s, which combines $\boldsymbol{a}_h$ from the HLC and $[\boldsymbol{p}, \boldsymbol{h}, b]$
  for each step in the episode **do**
    Select action $a = \mu(s|\boldsymbol{\theta}^l) + \mathcal{N}$ according to the current policy and exploration noise
    Compute r^l as the difference in portfolio value
    Store transition (s, a, r^l, s') in R
    Sample a minibatch of transitions (s, a, r^l, s') from R
    Set $y = r^l + \gamma Q'(s', \mu'(s'|\boldsymbol{\theta}^{l'}))$ for the minibatch
    Update the critic by minimizing the loss: $L = \frac{1}{N}\sum(y - Q(s, a|\boldsymbol{w}^l))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\boldsymbol{\theta}^l} J \approx \frac{1}{N} \sum \nabla_a Q(s, a|\boldsymbol{w}^l)|_{s,a=\mu(s|\boldsymbol{\theta}^l)} \nabla_{\boldsymbol{\theta}^l} \mu(s|\boldsymbol{\theta}^l)|_s$$

    Update the target networks:

$$\boldsymbol{w}^{l'} \leftarrow \tau \boldsymbol{w}^l + (1 - \tau)\boldsymbol{w}^{l'}, \quad \boldsymbol{\theta}^{l'} \leftarrow \tau \boldsymbol{\theta}^l + (1 - \tau)\boldsymbol{\theta}^{l'}$$

  end for
end for

Algorithm 6 Phased Alternating Training for HRT Agent

Initialize HLC policy network $\pi(a|s, \boldsymbol{\theta}^h)$ and value network $V(s|\boldsymbol{w}^h)$.
Initialize LLC actor network $\mu(s|\boldsymbol{\theta}^l)$ and critic network $Q(s, a|\boldsymbol{w}^l)$.
Phase 1 - HLC Training:
for episode $= 1$ to $EHLC$ **do**
    Focus on training the HLC with alignment rewards exclusively.
end for
Phase 2 - LLC Training:
for episode $= 1$ to $ELLC$ **do**
    Freeze HLC parameters.
    Concentrate on optimizing trade execution by the LLC using the augmented state information.
end for
Phase 3 - Alternating Refinement:
while not converged **do**
    Engage in alternating training between the HLC and LLC, gradually diminishing the frequency of switching as convergence approaches.
    Enhance HLC training by incorporating LLC rewards as specified in **Equation 3.2**, with the influence of LLC rewards increasing exponentially over time.
end while

3.4 Performance Evaluations

3.4.1 Stock Data Preprocessing

Expanding beyond the scope of the Dow Jones 30 constituent stocks as discussed in [17], [66], our thesis encompasses a broader pool: the S&P 500. This choice ensures adequate liquidity and uses the index as a benchmark for assessing overall U.S. market performance. The in-sample period, from 01/01/2015 to 12/31/2019 is for training, and the following year 01/01/2020 to 12/31/2020, is allocated for validation. In 2021, the U.S. equity market was bullish, characterized by strong gains across major indices, driven by economic recovery optimism and continued monetary support from the Federal Reserve. In contrast, 2022 was a bearish year for U.S. equities, with markets facing significant declines due to rising inflation concerns, tightening monetary policies, and geopolitical tensions. To test our models' performance on different market conditions, we perform the trading on both 01/01/2021 to 12/31/2021 and 01/01/2022 to 12/31/2022. We download our stock data from Yahoo Finance [3], which includes Open, High, Low, Close, and Volume (OHLCV) data. The Volume Weighted Average Price (VWAP) is derived daily from OHLCV data.

For constructing the High-Level Controller (HLC)'s state space, we compute the daily forward return of a stock on day T as the percentage change in its opening price from day T to day $T + 1$ using historical data. This process employs only the encoder part of the Transformer model [4], linking the encoder's output directly to a linear layer. Although

[3] https://finance.yahoo.com/

similar to traditional many-to-one RNNs, this model incorporates a self-attention mechanism. We adopted supervised learning to train the model with 158 features provided by Qlib[4], an open-source, AI-oriented quantitative investment platform, setting a lookback window of 10 trading days. For sentiment analysis, we utilize the finetuned FinGPT model [59] available on HuggingFace[5], which underwent instruct tuning on the LLaMA 2 13B model. Trading executions are assumed to occur at the market opening at 9:30 AM, incorporating the market's overnight reaction to news into the decision-making process. Thus, sentiment scores are calculated using the random sample of size 10 from previous day's 24 hours news, with a scoring system where positive sentiment is marked as 1, neutral as 0, and negative as -1, based on a simple average from outputs of large language models (LLMs).

3.4.2 Training Setup

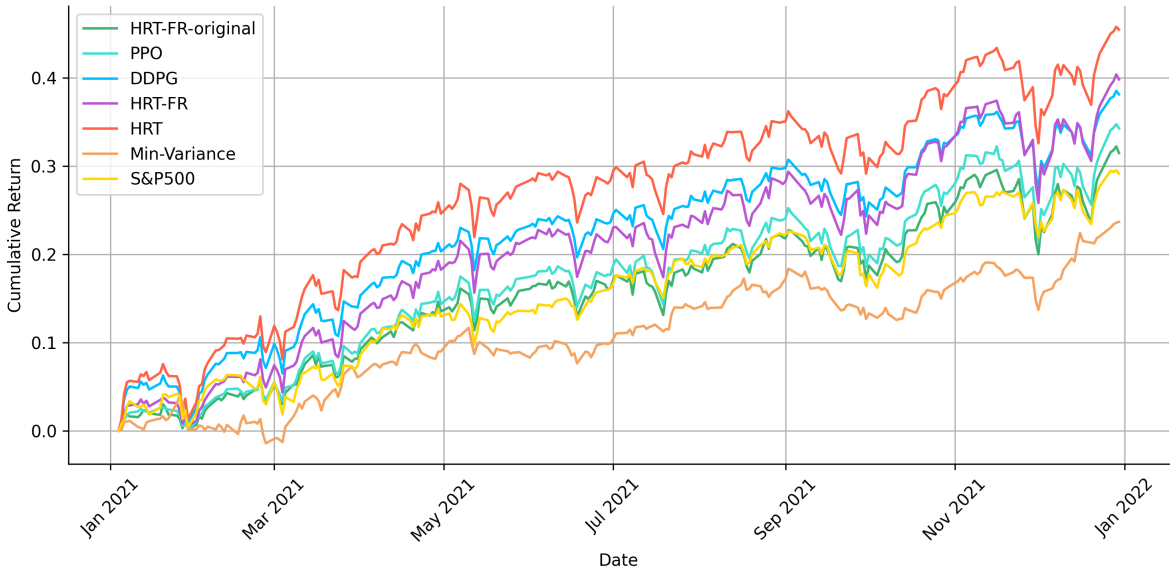
Our experiments leveraged NVIDIA Tesla V100 GPUs, utilizing the FinRL library[6] [75] to ensure comparability with prior work and efficiency in development. For the PPO-based HLC, we set the learning rate to 3e-4 and the clip parameter to 0.2, while the DDPG-based LLC was configured with a learning rate of 1e-3 for both the actor and critic networks, and a soft update parameter τ of 0.005. Both controllers used a replay buffer of 2e5, a batch size of 256, and a discount factor γ of 0.99, utilizing the AdamW optimizer over 5e5 total timesteps. Portfolio parameters included an initial capital of \$1,000,000 and a transaction cost percentage of 0.1%, with daily rebalancing. The total time cost for training the HRT agent was approximately 30 hours, reflecting the complexity of the model and the depth of data being processed. In terms of potential multiple sources of randomness such as model initialization, data mini-batch shuffle during training, and random sampling of news to compute sentiment scores, etc., we conducted ten independent experiments under the same setting with respect to different random seeds.

3.4.3 Evaluation Metrics

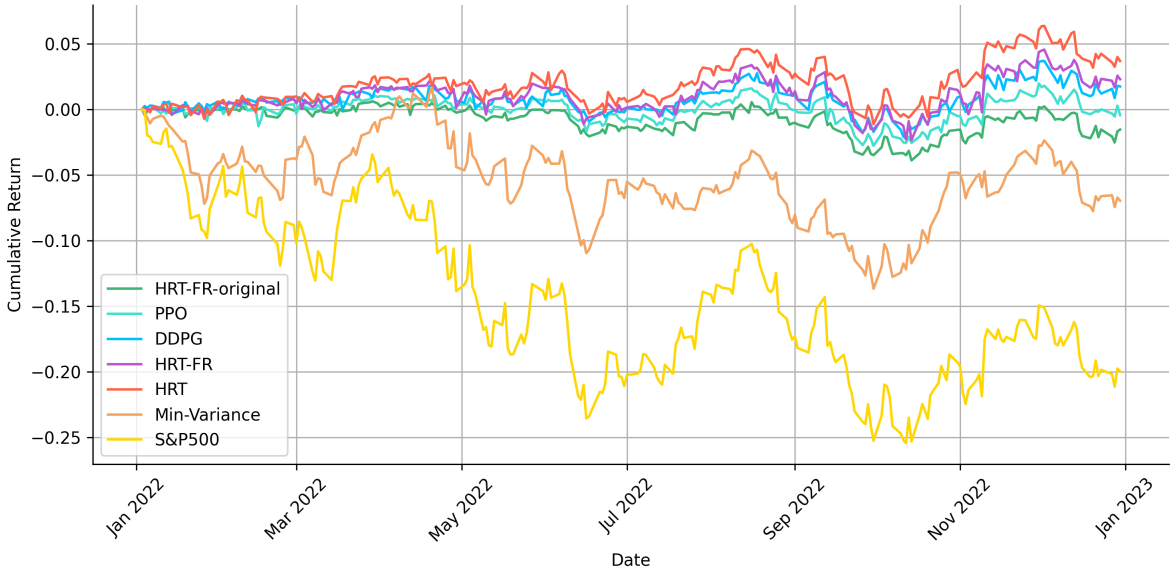
For a comprehensive comparison of the trading performance across different agents and benchmarks, we utilize the following metrics:

- Cumulative return: Aggregates the sum of daily returns up to the current day, offering a straightforward representation of total return.
- Annualized return: $(1 + \text{Cumulative Return})^{\frac{1}{n}} - 1$, where n represents the number of years over the return period. It provides a normalized measure of return, facilitating comparisons over varying time frames.
- Annualized volatility: Defined as $\sigma_p \times \sqrt{252}$, where σ_p denotes the standard deviation of daily returns. This formula adjusts volatility to an annual scale, considering the typical 252 trading days in a year, to standardize risk assessment.

[4] https://github.com/microsoft/qlib/blob/main/qlib/contrib/data/handler.py
[5] https://huggingface.co/FinGPT/fingpt-sentiment_llama2-13b_lora
[6] https://github.com/AI4Finance-Foundation/FinRL



(a) Year 2021 (bullish market)



(b) Year 2022 (bearish market)

Figure 3.3: Cumulative return curves of different investment strategies and S&P 500. Values are computed as the mean of ten independent training experiments, each with a different random seed.

- Sharpe ratio: Expressed as $\frac{R_p - R_f}{\sigma_p}$, with R_p being the portfolio return, R_f the risk-free rate (assumed to be 0 in this chapter), and σ_p the annualized volatility. The Sharpe ratio quantifies the risk-adjusted performance of an investment relative to a risk-free asset, highlighting the excess return per unit of risk.

- Max drawdown: Measures the largest percentage drop in portfolio value from a peak

to a subsequent trough before reaching a new peak. This metric is crucial for assessing the portfolio's risk and resilience to market downturns.

Table 3.1: Comparative Evaluation of Performance. The error terms represent the standard deviation calculated across ten independent experiments, each with a different random seed.

(a) Year 2021 (bullish market)

Metric	HRT-FR-original	PPO	DDPG	HRT-FR	HRT	Min-Var	S&P500
Cumulative Return	0.3147 ± 0.010	0.3428 ± 0.009	0.3813 ± 0.008	0.3983 ± 0.007	0.4548 ± 0.008	0.2368	0.2913
Annualized Return	0.3200 ± 0.009	0.3486 ± 0.009	0.3879 ± 0.007	0.4051 ± 0.007	0.4628 ± 0.008	0.2406	0.2961
Annualized Volatility	0.1489 ± 0.010	0.1498 ± 0.008	0.1458 ± 0.005	0.1665 ± 0.009	0.1687 ± 0.009	0.0980	0.1303
Sharpe Ratio	2.1494 ± 0.156	2.3274 ± 0.138	2.6601 ± 0.103	2.4336 ± 0.138	2.7440 ± 0.154	2.4549	2.2736
Max Drawdown	-0.0738 ± 0.018	-0.0808 ± 0.010	-0.0651 ± 0.013	-0.0845 ± 0.010	-0.0755 ± 0.016	-0.0516	-0.0521

(b) Year 2022 (bearish market)

Metric	HRT-FR-original	PPO	DDPG	HRT-FR	HRT	Min-Var	S&P500
Cumulative Return	-0.0154 ± 0.008	-0.0045 ± 0.004	0.0174 ± 0.005	0.0229 ± 0.005	0.0368 ± 0.004	-0.0696	-0.1995
Annualized Return	-0.0156 ± 0.007	-0.0045 ± 0.003	0.0176 ± 0.005	0.0232 ± 0.005	0.0372 ± 0.005	-0.0704	-0.2016
Annualized Volatility	0.0586 ± 0.008	0.0646 ± 0.007	0.0790 ± 0.008	0.0893 ± 0.005	0.0901 ± 0.005	0.1513	0.2416
Sharpe Ratio	-0.2664 ± 0.050	-0.0701 ± 0.037	0.2224 ± 0.059	0.2594 ± 0.045	0.4132 ± 0.048	-0.4654	-0.8344
Max Drawdown	-0.0448 ± 0.009	-0.0431 ± 0.008	-0.0484 ± 0.007	-0.0554 ± 0.008	-0.0548 ± 0.009	-0.1507	-0.2543

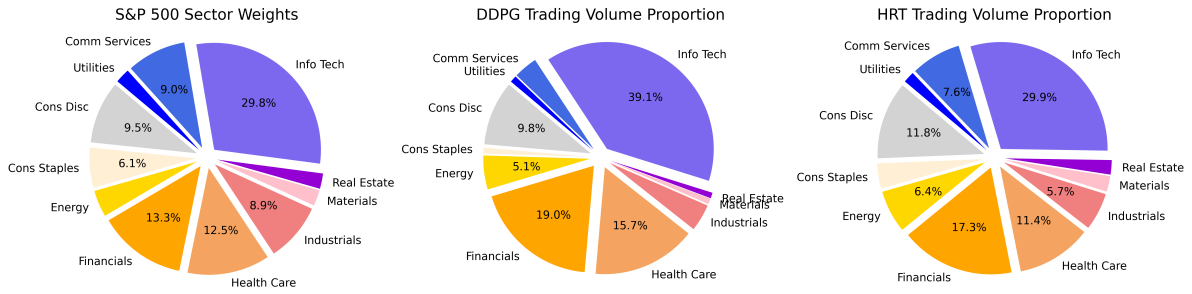


Figure 3.4: Comparison of Trading Volume Proportions: DDPG versus HRT. Values are computed as the mean of ten independent experiments, each with a different random seed.

3.4.4 Results

Enhanced Trading Performance

To assess the efficacy of various Deep Reinforcement Learning (DRL) trading agents, we monitored the cumulative returns of portfolios throughout the years 2021 and 2022. To ensure a balanced comparison, we benchmarked these performances against the S&P 500 and a minimum-variance portfolio, which was optimized daily for minimal variance using PyPortfolioOpt[7]. Additionally, we evaluated our Hierarchical Reinforced Trader (HRT), which incorporates predicted forward returns and sentiment scores within its HLC, with two variants: the first, denoted as HRT-FR, exclusively includes the predicted forward return $s^h = [\mathbf{fr}]$ with 500 input dimensions; the second, denoted as HRT-FR-original, incorporates

[7] https://github.com/robertmartin8/PyPortfolioOpt

the 158 original features from Qlib, as discussed in **Section 3.4.1**, for predicting forward returns, resulting in a state space of $158 \times 500 = 79,000$ input dimensions.

As detailed in **Figure 3.3** and **Table 3.1**, our HRT agent consistently outperformed both the standalone DDPG and PPO models, as previously explored in research [17], [66]. It achieved a Sharpe ratio of 2.7440 compared to the S&P 500 baseline of 2.2736 during the bullish year of 2021. In the bearish first half of 2022, characterized by significant market volatility and downturns, as evidenced by the S&P 500's performance, DDPG, HRT-FR, and HRT maintained minor but positive cumulative returns with significantly lower drawdowns, demonstrating their ability to adeptly navigate complex market dynamics. Notably, our HRT agent achieved a Sharpe ratio of 0.4132 during this period, with a significantly smaller drawdown compared to the S&P 500 portfolio.

Furthermore, HRT-FR, which only uses forward returns in the HLC, achieved respectable results in both years but did not outperform the standard HRT. This outcome underscores the value of incorporating predicted sentiment scores. Conversely, HRT-FR-original, with its high-dimensional state space, performed the poorest among the DRL agents, failing even to outperform the S&P 500 in 2021. We hypothesize that the challenge lies in training on such a high-dimensional input state and generalizing effectively to out-of-sample periods. In summary, the results from both years affirm the robustness of our standard HRT agent in balancing risk and return to generate a more profitable portfolio.

Alleviating DRL Challenges in Multi-Stock Trading

In addressing the challenges of deploying Deep Reinforcement Learning (DRL) agents for multi-stock trading, our Hierarchical Reinforced Trader (HRT) showcases significant improvements over the issues discussed in **Section 3.1**. By bifurcating the action space into two manageable components, HRT effectively navigates the dimensional complexity challenge. The action space for the High-Level Controller (HLC) narrows to 3^N, focusing solely on trading directions and bypassing the need to quantify trades, thus streamlining the decision-making process. Empirical evidence indicates that approximately 80% of stocks necessitate decisions on precise trading volumes, further compressing the Lower-Level Controller's (LLC) action space.

For the analysis of the inertia or momentum effect, we traded stocks from the DJIA 30 across the years 2021 and 2022. The trading heatmaps for both the standalone DDPG, as discussed in a previous study [66], and our Hierarchical Reinforced Trader (HRT) are depicted in **Figure 3.1**. These visualizations reveal a consistent trend toward more diversified and frequent trading under varying market conditions. Although only DIJA 30 stocks was selected to enhance the clarity of the heatmaps, we observed analogous trends with S&P 500 stocks. We believe HRT's ability to conduct more diversified and frequent trading can be attributed to the HLC state space, which is driven by the latest predictive returns and recent sentiment trends. Its alignment reward is set to zero if the HLC decides to hold a stock, encouraging more diversified and frequent trading. It is then passed to the LLC to determine the optimal execution amount.

Addressing the challenge of insufficient diversification, we compared the sector-based trading volume proportions within our portfolios against the average sector weights of the S&P 500 throughout the trading period. Sector weights within the S&P 500 reflect the

relative market capitalization of each sector, averaged over the trading period to establish a baseline for comparison. The comparative analysis, visually represented in **Figure 3.4**, shows that the DDPG portfolio exhibits a pronounced deviation from the S&P 500 sector weights, with a significant focus on sectors such as Information Technology, Financials, and Health Care, while minimally engaging with sectors like Consumer Staples, Real Estate, and Utilities. Conversely, our HRT system, despite also favoring trades in predominant sectors, demonstrates an alignment closer to the S&P 500's average sector weights. This not only indicates adherence to prevailing market valuations and trends but also suggests that the HRT portfolio is diversified in a manner akin to the index itself. The sector exposure of our portfolio mirrors the broader market distribution, endorsing inherent diversification across various sectors. This diversification is corroborated by the varied trading activity within the HRT portfolio, as seen in **Figure 3.1**, underscoring our system's ability to maintain a balanced sector distribution that mirrors prevailing market trends.

Chapter 4

Improved Univariate Flagging Algorithm (UFA): In a Broader Generalized Additive Model (GAM) Prospective

Currently, many statistical learning models are over-parameterized and difficult to interpret, which is unfriendly to practitioners such as doctors and lawyers. The recently proposed Univariate Flagging Algorithm (UFA) [76] makes progress toward finding interpretable univariate algorithms. However, it has strong assumptions and is restricted to simple binary classification cases. In this chapter, we propose an improved UFA framework under the Generalized Additive Model (GAM) that can model any data generated by exponential family distributions. We compared both the improved and original UFA on different public benchmark datasets. As a result, our framework not only achieves better predictive results than previous methods but also retains the excellent robustness of the previous UFA against missing and imbalanced datasets. We further achieve an average 3.2% accuracy improvement compared to the original UFA version's decent results.

4.1 Introduction

Machine learning methods have proven highly effective in extracting critical information from data across various application areas. Recent advancements include the development of deep learning models that incorporate hundreds of layers and diverse ensemble techniques. For example, Convolutional Neural Networks (CNNs) [77] and Transformers [4] can autonomously identify complex relationships within data, achieving higher accuracy in certain tasks than traditional methods. However, the adoption of these classifiers is often hindered by their lack of transparency. Practitioners, especially in healthcare, need clear explanations of how models make decisions from data to support clinical diagnoses. This necessity becomes even more pronounced as the complexity of models and the presence of missing or noisy data can undermine the reliability of machine learning applications.

One possible solution is to maintain "black-box" models but attempt to interpret them, which led to the field of explainable machine learning (XML) — a field focused on understanding and interpreting the behavior of machine learning models. Global explainable

techniques, such as partial dependence plots (PDP), show the marginal effect one or two features have on the predicted outcome of a machine learning model [78]. Local interpretation methods explain individual predictions; for example, the Local Interpretable Model-Agnostic Explanations (LIME) technique trains local surrogate models to explain individual predictions [79]. SHAP (SHapley Additive exPlanations) explains individual predictions based on the Shapley values in game theory [80]. From a computational perspective, different runs of LIME produce slightly different explanations due to the randomness in sampling local data points, and computing SHAP values is not scalable for datasets with a large number of features.

Another possible approach to tackling the problem of inexplicability is to use simple but effective models. For example, simple univariate discriminant analysis, which makes no use of covariance matrices, has performed similarly to much more sophisticated and popular machine learning methods [81]. Methods like Interpretable Decision Sets consist of sets of independent if-then rules. Because each rule can be applied independently, decision sets are simple, concise, and easily interpretable [82]. Other interpretable methods, following the idea of decision trees, convert features into optimal cutpoints based on objectives, e.g., a minimum p-value approach for finding optimal cutpoints for binary classifications [83]. The Patient Rule Induction Method (PRIM) procedure identifies rectangular subregions of the feature space associated with a high (or low) likelihood of the outcome, though users must clearly define the "optimal" subregion by specifying the preferred trade-off for the application.

Recently, the Univariate Flagging Algorithm (UFA) demonstrated automatic cutpoint detection with less user input in a univariate setting. At its most basic level, UFA finds the value x_{opt} univariately that maximizes the difference in the outcome rate for observations that fall outside x_{opt} and a baseline rate (e.g., rate of the interquartile range, defined as the 25^{th} to 75^{th} percentile) while maintaining a good level of support [76]. It achieves similar predictive accuracy compared to more sophisticated machine learning algorithms while providing easily interpretable results and stability against missing and noisy data. UFA fits clinical data very well since values outside clinically defined thresholds are associated with high mortality, while values within normal ranges may not provide much information. Many biomedical indicators fall within certain ranges.

However, the current UFA has some major limitations. [76] only demonstrated their algorithm using or converting to relatively simple binary classification cases. The current UFA only considers continuous explanatory variables. Moreover, the assignment of equal weight to each variable is clearly not ideal. In this chapter, we propose an improved Univariate Flagging Algorithm (UFA) version with broad applications in multiclass classification and regression tasks from a Generalized Additive Model (GAM) perspective [84], which could be used to model any data generated by exponential family distributions. We then prove that the original UFA version is a special case of GAM. Finally, we demonstrate that our method performs better than the original UFA on different benchmark datasets.

4.2 Methods

4.2.1 Original Univariate Flagging Algorithm (UFA)

The original Univariate Flagging Algorithm (UFA) [76] extracts a clear decision rule for each variable identified as significant. The rules take the simple form:

For single [feature/variable], a value [above/below] [threshold] is associated with a significantly [higher/lower] incidence of outcome [Y].

For downstream binary classification tasks (where the label y takes either 0 or 1), UFA creates an indicator variable or "flag" for each significant feature threshold. This flag takes the value of one if the variable's value exceeds the threshold and zero otherwise. It then sums up the flags for each data sample. Some flags are related to the label **0**, others to the label **1**. What makes the original UFA interpretable is that it distills information from potentially thousands of independent variables into two dimensions, making the results easy to visualize [1].

To better illustrate our improvement on the original UFA, we formalize the algorithm in **Algorithm 7**. UFA needs to conduct $O(nm)$ statistical tests in the training phase to identify the optimal thresholds, where n is the number of variables and m is the number of potential thresholds per variable. What makes UFA efficient is that it can perform these tests in parallel with minimal user settings.[2] We will discuss our extensions in detail in the following four subsections.

[1] For details, please refer to the N-UFA classifier in the original UFA paper. [76]

[2] Users only need to specify the significance level.

Algorithm 7 Original Univariate Flagging Algorithm (UFA) on binary classification

Input: Feature $x_1, \cdots, x_n$ [3], binary label y
Output: 2 Dims tuple: (Number of **1** flags, number of **0** flags)
Step 1. Find optimal thresholds
for Each feature x_i **do**
  Select threshold low/high candidates l_{ij} and h_{ij} $(n = 50)$
  $l_{i1}, \cdots, l_{i50} \in (min(x), median(x))$, $h_{i1}, \cdots, h_{i50} \in (median(x), max(x))$[4]
  for Each threshold l_{ij} and h_{ij} **do**
    if Support$(l_{ij}) \geq 5$ or Support$(h_{ij}) \geq 5$ **then**
      (Only show test on l_{ij}. h_{ij} is similar)
      Conduct binomial proportion test with $H_{0ij} : m_i = n_{ij}$
      m_i, n_{ij} is the label proportion defined as:
      $m_i = P(y = 1 | x_i \in (x_i^{q25}, x_i^{q75}))$
      $n_{ij} = P(y = 1 | x_i \in (x_i^{min}, l_{ij}))$
      Compute test statistics z_{ij} and p value p_{ij} [5]
      if p_{ij} is significant **then**
        return Optimal threshold $l_i^* = argmax(|z_{ij}|)$
      else
        return None
      end if
    end if
  end for
end for
Step 2. Convert l_i^*, h_i^* to flags
for Each feature's l_i^*, h_i^* **do**
  (Only show test on l_i^*. Case h_i^* is similar)
  if l_i^* not None and $P(y = 1 | x_i \leq l_i^*) \geq$ average proportion of target 1 between (x_i^{q25}, x_i^{q75}) **then**
    $f_i^l = 1$
  else
    $f_i^l = 0$
  end if
end for
Step 3. Summarize features on the sample level
for Each data sample **do**
  return $(\sum(\mathbb{1}(f_i^l = 1) + \mathbb{1}(f_i^h = 1)), \sum(\mathbb{1}(f_i^l = 0) + \mathbb{1}(f_i^h = 0)))$
end for

[3] UFA may work better for features distributed normally. Users can do feature transformations to achieve better normality.

[4] Use maximum or minimum values may be affected by outliers.

[5] Please refer to original UFA paper [76] for details.

4.2.2 Extension 1: Extending the Framework for Categorical Variables

The original UFA [76] primarily addresses binary classification cases with continuous features. However, categorical features such as gender play a crucial role in predicting outcomes in various domains. For instance, gender significantly impacts breast cancer mortality rates [85], and the presence or absence of influenza types A and B can provide valuable insights for detecting COVID-19 [86]. Disregarding these features could result in a substantial loss of information. To incorporate categorical features, we propose updating the binomial proportion test to a Chi-square (χ^2) test in **Algorithm 7**.

For a categorical feature with K possible values $v_k, k = 0, \cdots, K$ and a label with R possible classes, we construct a $K \times R$ contingency table. We then use Pearson's Chi-square (χ^2) test for independence as shown in **Equation 4.1**:

$$T = \sum \frac{(O - E)^2}{E} \sim \chi^2(df), \quad df = (K - 1) \times (R - 1) \tag{4.1}$$

where O represents the observed frequency in the contingency table, E the expected frequency, and df the degrees of freedom. If the p-value is significant under the specified significance level, we can obtain flags for each possible value $v_j \in (v_1, \cdots, v_{k_i})$ of feature x_i, as demonstrated in **Equation 4.2**. This approach is analogous to summing up the number of flags f_i^v, similar to f_i^l and f_i^h in a binary classification:

$$f_i^{v_j} = \begin{cases} 1 & \text{if } P(y = 1 \mid x_i = v_j) \geq \text{global average proportion of } y = 1, \\ 0 & \text{otherwise.} \end{cases} \tag{4.2}$$

This enhancement enables our framework to handle classification tasks involving both categorical and continuous features within a univariate flagging setting.

4.2.3 Extension 2: Extending UFA to Regression Tasks

For continuous labels y in regression tasks, we substitute the binomial proportion test with a T-test in line 14 of **Algorithm 7**. Instead of taking the label proportion, we compute the sample averages m_i and n_{ij} as defined in the null hypothesis for each candidate threshold l_{ij}. (h_{ij} would be similar.) These averages for continuous features are calculated using **Equation 4.3** and **4.4**.

$$m_i = \text{mean}(x_i | x_i \in (x_i^{q25}, x_i^{q75})) \tag{4.3}$$

$$n_{ij} = \text{mean}(x_i | x_i \in (x_i^{min}, l_{ij})) \tag{4.4}$$

Flagging features can result in a significant loss of information in regression tasks. Therefore, in our regression setting, we omit the second step in **Algorithm 7** that converts cutpoints to flags. For continuous features, we use the optimal lower and upper cutpoints l_i^*, h_i^* as knots to build linear splines. These splines are constructed by fitting a line within each subregion of the predictor space defined by the knots, ensuring continuity at each knot. For categorical features, we use their total k_i possible values $(v_1, v_2, \cdots, v_{k_i})$ as knots to construct

step functions. Each univariate feature is modeled separately, as shown in **Equation 4.5** and **4.6**.

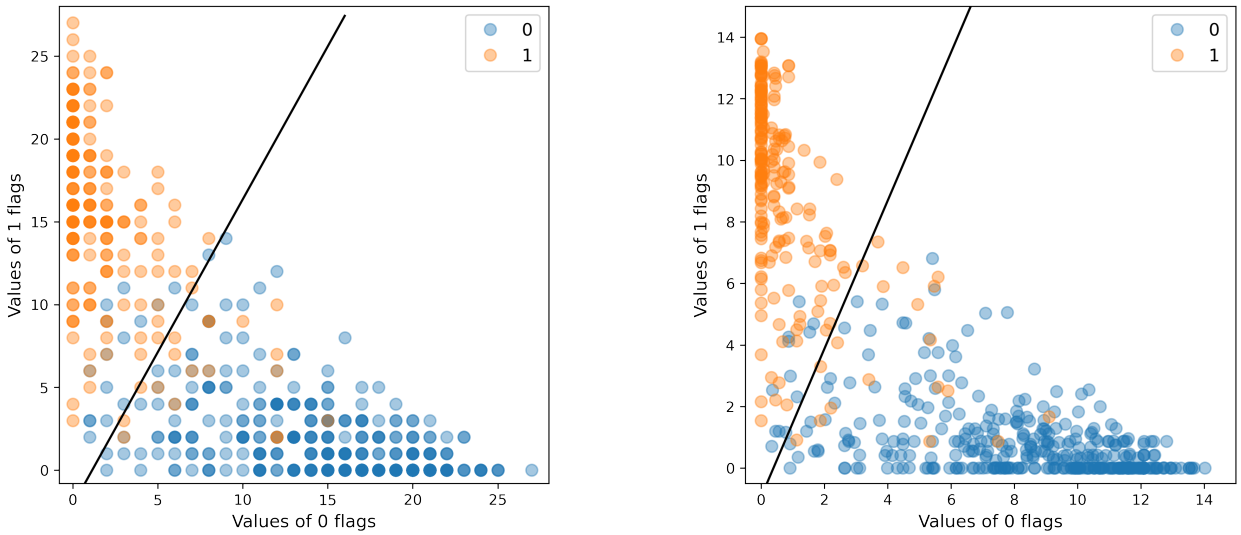
$$f_i(x_i)_{con} = \beta_0 + \beta_i(l_i^* - x_i)_+ + \beta_i'(x_i - h_i^*)_+ \tag{4.5}$$

where $(x-y)_+ = max(0, x-y)$. We assume that only values of a feature below the l_i^* or above the h_i^* will influence the outcome. Within $[l_i^*, h_i^*]$, we assume a constant level of β_0.

$$f_i(x_i)_{cat} = \alpha_0 + \sum_{j=1}^{k_i} \alpha_j \mathbb{1}(x_i = v_j) \tag{4.6}$$

4.2.4 Extension 3: Assigning Different Weights When Summing Up Features

In **Algorithm 7** line 34, each flag receives an equal weight of one when summing up different features. Since different features contain varying levels of predictive power, one possible way to assign weights is by constructing correlation measurements between each feature and the label. This approach not only boosts accuracy but also reduces overlap during visualization, as shown in **Figure 4.1**. We present different scenarios based on varying feature and label types, as illustrated in **Table 4.1**.



(a) Original UFA

(b) Improved UFA (weighted)

Figure 4.1: Summing up flags to two dimensions on the Wisconsin Breast Cancer Dataset. [87]

Pearson's correlation coefficient measures the linear correlation between two sets of continuous data, calculated by the covariance of the data sets divided by the product of their standard deviations, as shown in **Equation 4.7**. The Point-Biserial correlation coefficient is used specifically for cases involving continuous features and a binary label [88]. Since the Point-Biserial correlation coefficient is mathematically equivalent to Pearson's correlation

Table 4.1: Weights under different scenarios.

	continuous y	categorical y
continuous x	Pearson's correlation coefficient $r \in [-1, 1]$	Pearson's correlation coefficient $\eta \in [-1, 1]$
categorical x	Pearson's correlation coefficient $\eta \in [1, 1]$	Cramer's V $v \in [0, 1]$

coefficient, we use Pearson's for the two cases involving continuous data. For cases involving both categorical data, we use Cramér's V to measure associations, which yields a value between 0 and 1 [89]. Cramér's V is computed by taking the square root of the chi-squared statistic T obtained in **Equation 4.1**, divided by the sample size and the smaller of $(K-1)$ or $(R-1)$, where K is the number of columns, and R is the number of rows of contingency table constructed by categorical feature and target. The formula for Cramér's V is defined in **Equation 4.8**. Finally, we assign each feature a normalized weight, $w_{\text{norm}} = \frac{|w|}{\sum |w|}$.

$$w_{\text{pearsonR}} = \frac{\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \overline{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \overline{y})^2}} \tag{4.7}$$

$$w_{\text{cramerV}} = \sqrt{\frac{T/n}{\min(K-1, R-1)}} \tag{4.8}$$

4.2.5 Extension 4: Integrating Everything into a Generalized Additive Model (GAM) Perspective

We now demonstrate that the original UFA and our extensions are special cases of the Generalized Additive Model (GAM) [84], culminating in a general framework for UFA.

A typical GAM relates the expected value of the label Y to predictor variables x_i. An exponential family distribution is specified for Y (e.g., Gaussian, Binomial, or Poisson distributions) alongside a link function g (e.g., identity or log) through the structure shown in **Equation 4.9**:

$$g(E(Y)) = \beta_0 + \sum_i f_i(x_i) \tag{4.9}$$

Each $f_i(x_i)$ may take on a parametric or non-parametric form, termed "smooth functions". The flexibility to allow non-parametric fits, with relaxed assumptions about the actual relationship between the response and predictors, provides potential for better fits to data than purely parametric models, albeit possibly at some loss of interpretability. Therefore, we only consider a simple parametric format in our extended UFA framework.

Regression case

For the regression tasks discussed in **Section 4.2.3**, instead of flagging the cutpoints, we decided to make our framework a non-flagging version for better predictive accuracy. We use **Equations 4.5** and **4.6** to model features univariately and set the link function $g(\cdot)$ to be the identity function $g(\mu) = \mu$. The model of Y is represented as the sum of the feature functions $f_i(x_i)con$ and $fi(x_i)_{cat}$.

Classification case

For classification tasks, a simple idea is to change the link function to $g(\mu) = \log(\mu/1-\mu)$ to model the discrete label Y in **Equation 4.10** like the non-flagging fashion as regression tasks.

$$\log(\frac{E(Y)}{1-E(Y)}) = \beta_0 + \sum_i f_i(x_i) \tag{4.10}$$

The original UFA[6] extends further to flag features[7] and summarize the flags into two dimensions, which are dominated by one type of label (e.g., 0 flags and 1 flags). The number of flag types should be equal to the number of classes (K) we want to classify. This approach distills information from thousands of independent variables into two or three dimensions in most cases when $K = 2, 3$, making the results easy to visualize. In a flagging setting, rather than using linear splines and step functions to model each $f_i(x_i)$, we use 0 or 1 valued flags to represent features as shown in **Equation 4.11** and **4.12**, setting the βs and αs of **Equation 4.5** and **4.6** to one.

$$f_i(x_i)_{con} = \mathbb{1}(l_i^* - x_i)_+ + \mathbb{1}(x_i - h_i^*)_+ \tag{4.11}$$

$$f_i(x_i)_{cat} = \sum_{j=1}^{k_i} \mathbb{1}(x_i = v_j) \tag{4.12}$$

If we want to assign different weights to features as described in **Section 4.2.4**, we multiply a constant factor w_{pearsonR} for each continuous $f_i(x_i)$, and w_{cramerV} for each discrete one. The model can then be expressed as shown in Equation (4.13):

$$Y = \sum w_{\text{pearsonR}} f_i(x_i)_{\text{con}} + \sum w_{\text{cramerV}} f_i(x_i)_{\text{cat}} \tag{4.13}$$

Next, we convert all features to flags using **Equation 4.2** for categorical features and line 26 in **Algorithm 7** for continuous features. The final output is a K-dimensional vector for classification involving K classes, where K is significantly smaller than the original number of features. Thus, we can also consider UFA as a method for feature representation to extract predictive signals from the original features. We could build sophisticated machine learning classifiers on top of the vector. However, we will demonstrate that even very simple linear classifiers like Pocket Algorithm or Support Vector Machine (SVM) with soft margin would work in **Section 4.3**.

In summary, the original UFA and our extensions are special cases of GAM. We construct our model by considering features univariately, omitting feature interactions to maintain interpretability. The key difference in processing continuous and categorical features lies in the use of cutpoints or discrete possible values to construct $f_i(x_i)$. We explore the potential application of UFA in both regression and classification tasks, noting that while we prefer to maintain a flagging setting for classification, the regression setting remains non-flagging. However, our extended UFA is a general framework under GAM that can model any data

[6] More concisely, the N-UFA classifier in [76]

[7] Step 3 in **Algorithm 7**.

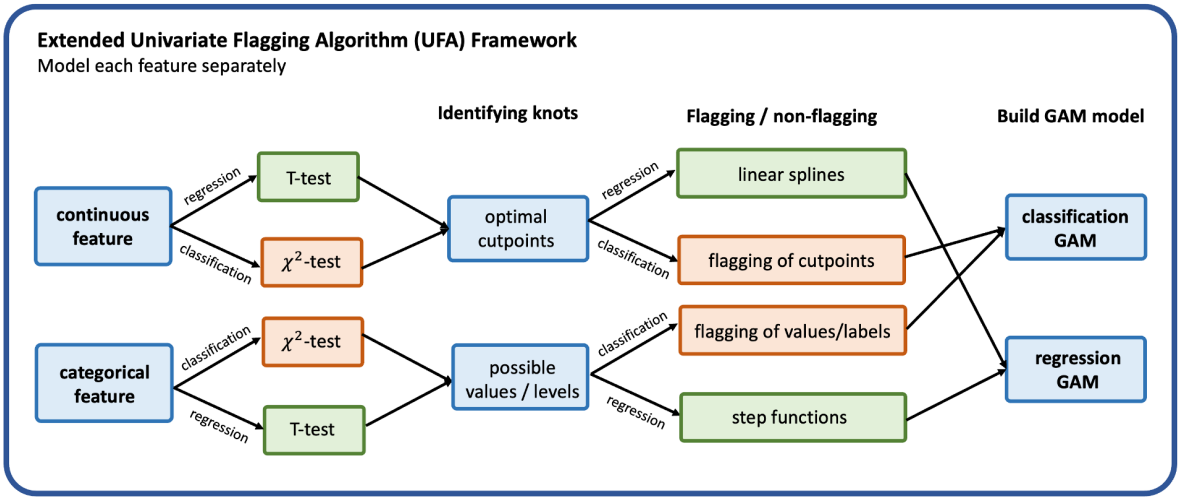


Figure 4.2: Overview of the extended UFA framework.

following exponential family distributions [84]. **Figure 4.2** provides an overview of our extended UFA framework.

4.3 Results

In clinical decision-making, doctors identify ranges of laboratory test values that may indicate a higher risk of developing or having a disease. However, the UFA framework is also applicable to non-clinical settings. In this section, we mainly focus on our applications in clinical datasets but will also discuss applications in other fields.

4.3.1 Identifying Optimal Cutpoints

UFA automatically detects target-defined separation thresholds in data. Similar to findings reported by [76], we demonstrate that the near-optimal thresholds identified by UFA align with scientific discoveries and have actual meanings. **Table 4.2** shows three examples of variables in the Cirrhosis Prediction Dataset [90] with known clinical thresholds. The prediction target is the histologic four-stage liver disease. We convert the problem into a binary classification problem: whether a patient is in the early two stages (1, 2) or in a later stage (3, 4).

For each variable in **Table 4.2**, the UFA-identified thresholds not only show directionality consistent with clinical understanding of cirrhosis, but the obtained thresholds also align with known physiological normal ranges as established by the National Institutes of Health. For patients violating these threshold ranges, the rate of entering the late stages (3 or 4) ranges from 58.4% to 65.3%, significantly higher than the overall rate for the septic population (35.8%).

Table 4.2: Examples of UFA-defined thresholds, Cirrhosis Prediction Dataset.

Variable	Normal Range	Threshold	Support	Rate of entering late stages
Copper [ug/day]	15-60	above 58	87	60.4%
Cholesterol [mg/dl]	<150	above 153	101	58.4%
Albumin [g/dl]	3.4-5.4	above 5.1	98	65.3%

4.3.2 Improvements on Classification Tasks

To demonstrate our improvement on UFA, we follow similar experiments as [76] did. **Table 4.3** summarizes the sample error rate comparison of UFA under three different benchmark classification datasets: Pima Indian Diabetes [91], Wisconsin Breast Cancer [87], and ALL/AML Cancer Datasets [92]. We also include a publicly available Heart Failure Prediction Dataset[8], which includes both continuous and categorical features. These datasets vary greatly in terms of complexity and target/non-target ratio. The reported results are the average of 1000 independent experiments performed for each setting. Considering the UFA classifier as a feature processing step that distills information from original feature dimensions to two dimensions in the following binary classification problem, we compare the UFA methods with Principal Component Analysis (PCA) using two principal components. We choose two powerful and interpretable methods for the downstream classifier: the Pocket Algorithm and the Support Vector Machine (SVM). We will also include the Random Forests classifier to compare results with the original UFA.

Table 4.3: Comparison of performance on different classification tasks.

Algorithms	Pima Indian Diabetes	Breast Cancer	ALL/AML Cancer	Heart Failure
UFA + RF[9] [76]	19.3±2.3	7.2±1.6	0.6±1.6	N.A
Improved UFA + RF	16.3±2.4	4.1±1.2	0.3±1.0	10.4±1.2
PCA(N=2) + RF	34.2±3.4	10.5±0.7	1.2±1.9	20.1±2.2
RF	16.2±1.3	6.5±1.8	0.0±0.4	11.3±1.3
Improved UFA + SVM	18.5±1.8	4.2±1.6	0.3±0.3	12.5±2.1
PCA(N=2) + SVM	35.6±2.2	8.1±0.4	1.5±1.0	20.7±1.6
SVM	17.8±2.0	2.1±1.4	0.2±0.6	16.2±0.6
Improved UFA + Pocket Algorithm	19.8±3.2	6.8±1.4	0.9±1.9	15.6±1.1
PCA(N=2) + Pocket Algorithm	30.2±2.0	13.4 ±0.8	2.0±1.4	23.2±1.4
Pocket Algorithm	23.1±0.3	8.1±0.4	1.4±0.5	18.5±1.2

Table 4.3 demonstrates that our improved UFA decreases the error rates by approximately 3.2% on average across all benchmark datasets, considering the confidence intervals. Moreover, its performance aligns with complex ensemble models, such as Random Forests. Considering UFA as a feature processing step, it performs much better than the PCA method when mapping features to two dimensions. UFA, with a simple linear classifier such as SVM (linear kernel) or Pocket Algorithm, is sufficient to generate decent accuracy. For the Heart Failure Dataset, our improved UFA achieves a 10.4% error rate, while it gets a 12.2% error

[8] https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction

rate with only continuous variables. Therefore, it is worthwhile to include those categorical features using the improved UFA.

4.3.3 Robustness to Missing and Noisy Data

To further test our improved UFA's ability to make predictions in the presence of missing and noisy data as the original UFA algorithm, we artificially introduced additional missing values and random deviations into the Heart Failure Prediction Dataset. **Table 4.4** compares the performance of our improved UFA with SVM to other common classifiers for the original Heart Failure Prediction Dataset and versions of the data where 25% and 50% of observations were randomly replaced with missing values. We observe that our improved UFA also enhances robustness to missing data, with accuracy decreasing by only 3.4% as the proportion of missing data increases to 50%.

Table 4.4: Comparison of different classifiers with varying amounts of missing data.

Algorithms	Error rate			AUC		
	0%	25%	50%	0%	25%	50%
Improved UFA + SVM	12.5±2.1	13.1±1.5	15.9±1.1	0.856±1.2	0.842±0.8	0.821±1.1
SVM	16.2±0.6	18.2±0.5	25.5±0.6	0.834±0.8	0.802±0.5	0.756±0.8
Random Forests (RF)	11.3±1.3	14.3±1.0	24.2±1.2	0.872±1.2	0.825±1.0	0.760±0.9

Table 4.5 provides similar results and compares the performance for the same dataset and versions of the data where 25% and 50% of observations were randomly perturbed by a value ϵ, distributed normally with mean zero and the empirical variance of the variable. We see that our improved UFA is also robust to inaccurate data, with accuracy decreasing by only 1.7% as the proportion of imprecise data increases to 50%, and its confidence interval is smaller than that of the Random Forests classifier.

Table 4.5: Comparison of different classifiers with varying amounts of imprecise data.

Algorithms	Error rate			AUC		
	0%	25%	50%	0%	25%	50%
Improved UFA + SVM	12.5±2.1	13.2±1.5	14.2±1.8	0.856±1.2	0.843±1.0	0.832±1.1
SVM	16.2±0.6	17.2±1.0	18.3±1.8	0.834±0.8	0.829±0.6	0.810±0.7
Random Forests (RF)	11.3±1.3	11.9±0.8	13.2±0.6	0.872±1.2	0.868±1.5	0.859±1.6

4.3.4 Potential to model extremely imbalanced labels

The original UFA was used to predict the rare event of landslides. In the dataset spanning nearly 13,000 days, only 2.3% recorded one or more landslides. This scenario is a common challenge in real-world applications, where datasets often display significant imbalances in positive outcomes, such as the number of likes after video exposure in recommendation systems or the total number of adverse events following vaccination. In this subsection, we apply

our improved UFA to the Credit Card Approval Prediction dataset [10]. This dataset, crucial for commercial banks, features a very low proportion of high-risk clients (only 13.23%). Accurately identifying these clients is essential for managing financial risk. We find that our improved UFA can also identify the optimal cutpoints of clients' features that imply a high potential risk, such as payment overdue. Moreover, subregions defined by a combination of cutpoints may enlarge the relative risk difference. **Figure 4.3** illustrates that clients are six times more likely to be high-risk when their total income is below $33,000 and their years of employment are fewer than eight.

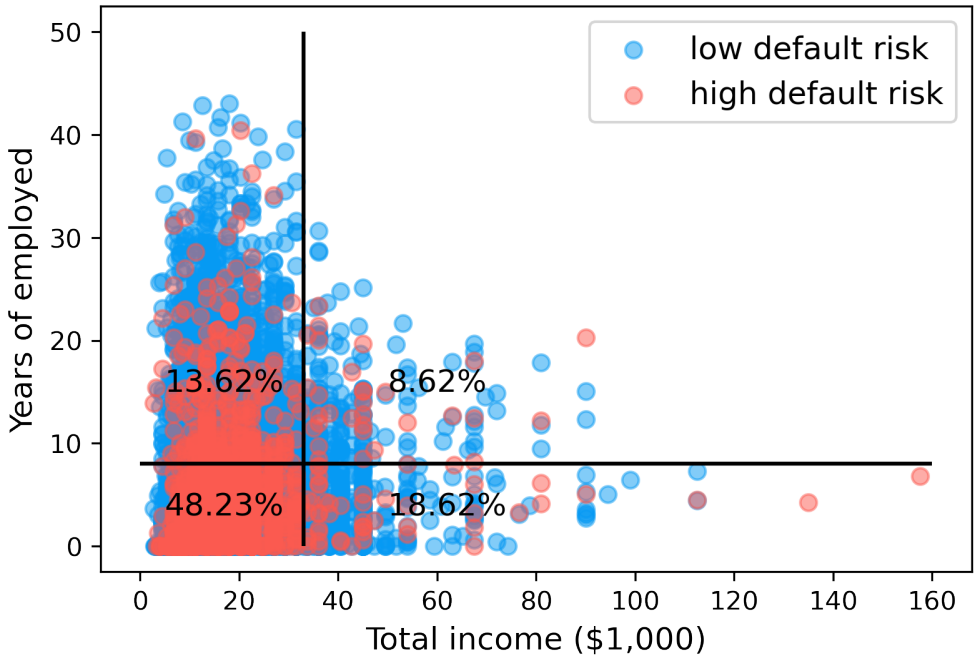


Figure 4.3: Credit card approval risk stratified by total income and years of employment.

Table 4.6: Comparison of performance on regression task.

Algorithms	RMSE	R^2
Improved UFA regressor	5.02±0.5	0.822±0.08
Linear Regression	5.16±0.9	0.762±0.10
Support Vector Regression (SVR)	4.23±0.6	0.882±0.02
Random Forest	4.34±0.5	0.890±0.05

4.3.5 Improvements on Regression Tasks

In this section, we utilize a non-clinical benchmark dataset, the Boston Housing Dataset [93], to predict the median value of owner-occupied homes. **Table 4.6** summarizes the predictive

[10] https://www.kaggle.com/datasets/rikdifos/credit-card-approval-prediction

performance using different regressors. It shows that our improved UFA, while sacrificing some accuracy for interpretability, incurs a slightly higher Root Mean Square Error (RMSE) than more sophisticated models such as Support Vector Regression (SVR) and Random Forest (RF). Despite this, we believe our improved UFA remains a potential candidate when addressing regression problems due to its interpretability.

Chapter 5

Conclusions and Future Work

In **Chapter 2**, we introduce a novel, adaptive retrieval-augmented LLMs framework integrating market feedback for dynamic weight allocation across different knowledge sources within the RAG module. Our goal is for our financial LLMs to not only extract the true underlying sentiment but also to produce sentiment outputs predictive of stock movements. We design a long feedback loop that refines the weights of the RAG module within the LLMs. Such adjustments affect the context analyzed by the LLMs, influencing the generated sentiment scores. These scores are then assessed against market feedback to further refine the model. Our extensive analysis of benchmark datasets demonstrates that aligning LLMs with market feedback—indirectly through the refinement of RAG weights—enhances both accuracy and F1 scores. The direct refinement approach, while straightforward, is notably effective, especially when employing RL-based methods. Further analysis of portfolio construction shows that LLMs' sentiment outputs can predict the next day's stock price movements. Both analyses indicate that our LLMs' sentiment outputs not only more accurately capture underlying sentiments but are also predictive of stock price movements.

We chose not to employ methods like reinforcement learning from human feedback (RLHF) [50] to guide the sentiment output of LLMs directly because it is challenging to design a reward function that accounts for the myriad factors influencing the entire stock market. It is also difficult for humans to ascertain whether a positive sentiment necessarily correlates with a positive market outcome even in the near term. Relying directly on market return signs as rewards might inadvertently focus LLMs on merely predicting price change directions based on textual data, overlooking the extraction of genuine sentiment within the text. Future research should consider the development of a suitable reward function, the training of an effective reward model (RM), and the application of RL methods for direct engagement and enhancement of financial LLMs.

In **Chapter 3**, we present the Hierarchical Reinforced Trader (HRT), a strategy that breaks down the trading process into two connected processes, aiming to boost trading performance by deeply understanding market dynamics and improving execution. We deploy a Proximal Policy Optimization (PPO)-based High-Level Controller (HLC) for selecting trading directions and a Deep Deterministic Policy Gradient (DDPG)-based Low-Level Controller (LLC) to decide the optimal number of shares to trade. We also introduce the Phased Alternating Training algorithm for simultaneous training of these two parts. In testing with actual S&P 500 data, our standard HRT agent consistently achieved positive cumulative re-

turns and maintained strong Sharpe ratios under various market conditions. Notably, even in bearish markets characterized by significant stock price declines and high volatility, the HRT agent managed to maintain a positive Sharpe ratio. Additionally, HRT shows promise in reducing the dimensions of actions and states, suggesting that future studies could look into separating the buying and selling actions to further narrow the action space. HRT also appears to mitigate the inertia or momentum effect, which typically limits diversification in models like DDPG, thus enhancing the profitability and robustness of trading algorithms and inviting a reevaluation of applications in multi-stock trading.

In **Chapter 4**, we present a broader framework for the Univariate Flagging Algorithm (UFA) from the Generalized Additive Model (GAM) perspective, compared to the original UFA that only discusses the binary classification case [76]. We demonstrate that our framework not only expands UFA's application scope but also achieves better results than the original UFA across various benchmark datasets, extending beyond the typical focus on UFA's clinical applications. Overall, our proposed UFA performs well with datasets that have small samples, missing values, and noisy data. It can be applied to unbalanced data to identify subregions with high risk. To interpret the model from a features perspective, we can rely on the cutpoints output by UFA to understand the relationship between features and labels. For data sample-level interpretation, we can explain the model's decision-making process using the summarized flags vector. In summary, we enhance the original UFA and offer a broader perspective on increasing model interpretability while maintaining predictive performance.

Let's revisit our proposed portfolio management framework as demonstrated in **Figure 5.1**. **Chapter 2** discusses how LLMs can be used to extract sentiment scores from financial textual data and how we build on a series of powerful financial LLMs. **Chapter 3** discusses how we use the transformer-based model to predict stock returns to construct the state space of the High-Level Controller (HLC). We then explore deep reinforcement learning, specifically hierarchical reinforcement learning in stock trading, to deliver better asset allocation and construct several profitable yet robust portfolios under different market conditions. **Chapter 4** presents advancements in the Univariate Flagging Algorithm (UFA) for improved model interpretability and predictive performance.

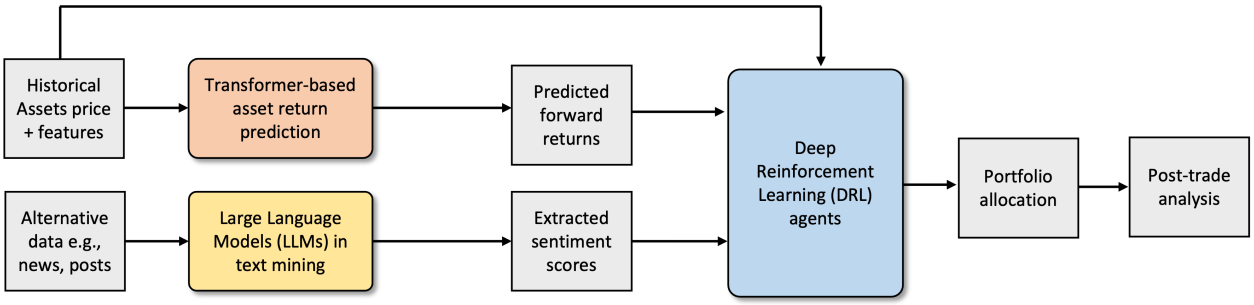


Figure 5.1: Revisit our proposed next-generation portfolio management framework.

In summary, we demonstrate the significant potential of integrating advanced AI technologies into finance, setting a new standard for intelligent portfolio management systems. Our research highlights the importance of state-of-the-art technologies and model interpretability in enhancing financial decision-making processes for portfolio management.

For future work, a current limitation of our financial LLMs is their dependence on keyword similarity for information retrieval, risking matches with documents that seem similar but are not semantically related. Future efforts could include developing a semantic search capability within our RAG module by generating vector representations of data for fast retrieval from vector databases. Although keyword searches are simpler, semantic search requires more complex engineering but promises a more effective way for LLMs to access pertinent information.

Another future direction for improving multi-stock trading and allocation could involve modeling the trading process as a Partially Observable Markov Decision Process (POMDP), as a few studies [16], [94] have done, taking into account the constraints imposed by the stock market. Investigating adaptive learning rates and experimenting with the most recent DRL models could also lead to improvements in overall performance.

In our current portfolio management framework, we only consider assets like constituent stocks of the S&P 500. However, a real portfolio could also include the entire market, or other assets like ETFs, bonds, and derivatives. These financial instruments have their own characteristics and risk-return trade-offs that should be considered. Expanding the asset universe to include these additional instruments would provide a more comprehensive and realistic approach to portfolio management, addressing a wider range of investor needs and market conditions.

Appendix A

Additional Figures and Tables

A.1 Instruction Set

Component	Description	Purpose
Current Weights	Weights assigned to various knowledge sources within the RAG module.	Indicates the current influence of each source in sentiment analysis.
Historical Accuracy Rate	The accuracy of sentiment predictions over a historical period, measured as a percentage.	Reflects the effectiveness of past predictions in aligning with market movements.
Average Overlap Coefficient	Measures the average similarity between the queries and the retrieved documents, calculated using metrics like cosine similarity.	Assesses the relevance of retrieved information to the query.
Historical Stock Return	The return generated by the stock portfolio over a specified historical period.	Provides insight into the past financial performance.
Technical Indicators	Includes indicators such as MACD, RSI, etc., used to evaluate market conditions.	Helps in identifying trends and potential market turnarounds.

Table A.1: Components of RL State

A.2 Detailed workflow example

A.3 State Construction of RL-based Weights Refinement

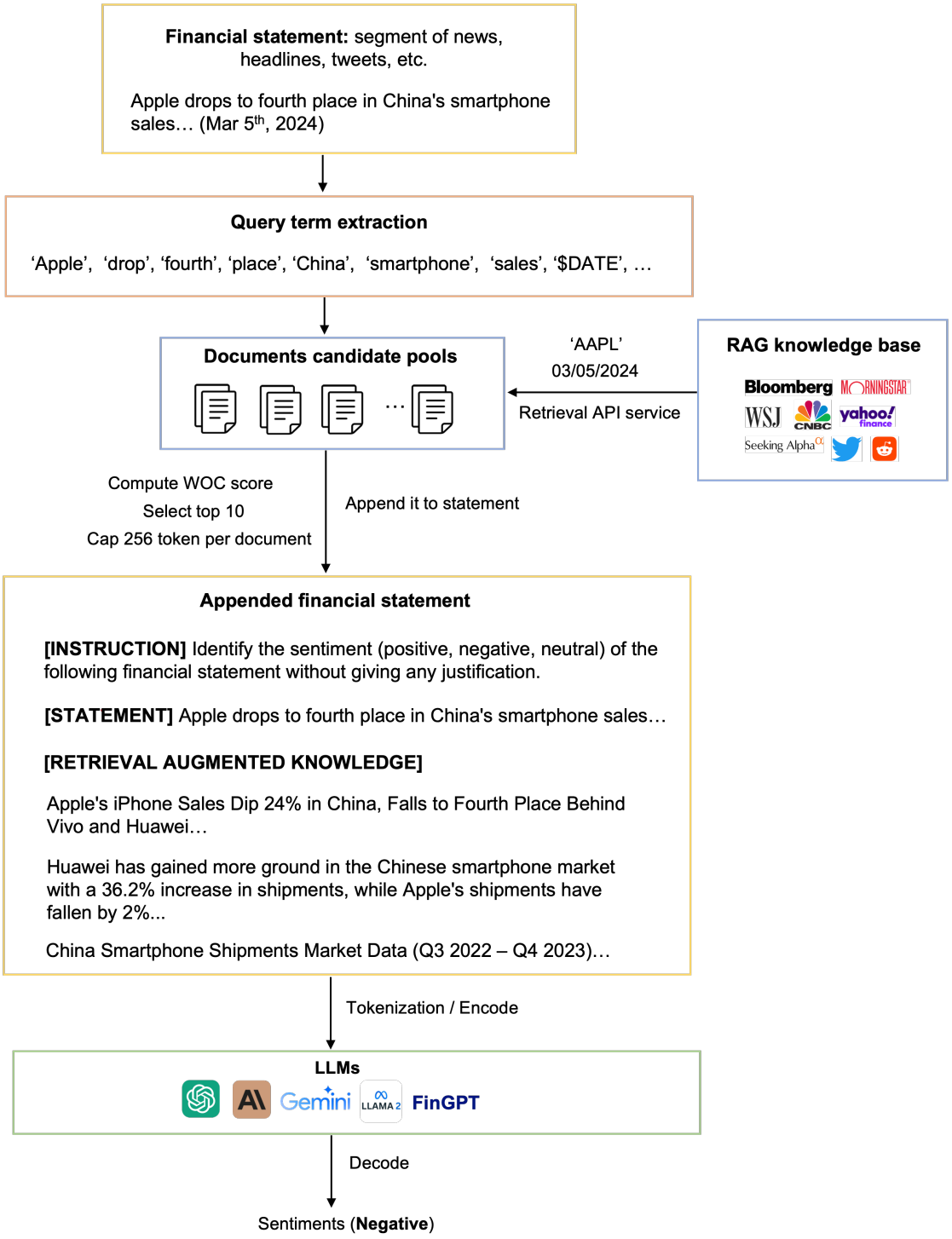


Figure A.1: Detailed illustrative example for our proposed workflow.

#	**Instruction**
1	Identify the sentiment (positive, negative, neutral) of the following financial statement without giving any justification.
2	Choose the appropriate sentiment: positive, negative, or neutral for the following financial news, and respond only with the sentiment category.
3	From the options positive, negative, or neutral, select the one that best fits the sentiment of the provided financial news.
4	Read the following financial statement and classify its sentiment as either positive, negative, or neutral. Provide only the classification.
5	Determine the sentiment of the following financial news—positive, negative, or neutral—and state your choice clearly.
6	Evaluate the sentiment of this financial statement as positive, negative, or neutral. Reply with your sentiment classification only.
7	Assign a sentiment category (positive, negative, or neutral) to the following financial news. Do not include any explanations.
8	For the financial statement below, indicate whether the sentiment is positive, negative, or neutral. Just provide the sentiment.
9	Categorize the sentiment of the provided financial news as positive, negative, or neutral. Only the sentiment category is required.
10	Based on the following financial news or statement, classify its sentiment as positive, negative, or neutral. Simply select your response from the options: positive, negative, or neutral, without providing any explanation.

Table A.2: Instructions for Classifying Financial News Sentiment

References

[1] H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.

[2] F. Advisor, *Modern portfolio theory*, Accessed: 2024-05-15, 2024. URL: https://www.forbes.com/advisor/investing/modern-portfolio-theory/.

[3] M. Kopp, "The impact of the ai revolution on asset management," *arXiv preprint arXiv:2304.10212*, 2023.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[5] S. Li, X. Jin, Y. Xuan, X. Zhou, W. Chen, Y.-X. Wang, and X. Yan, "Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting," *Advances in neural information processing systems*, vol. 32, 2019.

[6] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," *Advances in Neural Information Processing Systems*, vol. 34, pp. 22 419–22 430, 2021.

[7] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, 2021, pp. 11 106–11 115.

[8] F. Xing, E. Cambria, and R. Welsch, *Intelligent asset management*. Springer, 2019.

[9] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[10] S. Wu, O. Irsoy, S. Lu, V. Dabravolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, and G. Mann, "Bloomberggpt: A large language model for finance," *arXiv preprint arXiv:2303.17564*, 2023.

[11] J. Yang, H. Jin, R. Tang, X. Han, Q. Feng, H. Jiang, B. Yin, and X. Hu, "Harnessing the power of llms in practice: A survey on chatgpt and beyond," *arXiv preprint arXiv:2304.13712*, 2023.

[12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[13] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[14] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," *arXiv preprint arXiv:1910.13461*, 2019.

[15] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, *et al.*, "Mastering the game of go with deep neural networks and tree search," *nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[16] T. Kabbani and E. Duman, "Deep reinforcement learning approach for trading automation in the stock market," *IEEE Access*, vol. 10, pp. 93 564–93 574, 2022.

[17] H. Yang, X.-Y. Liu, S. Zhong, and A. Walid, "Deep reinforcement learning for automated stock trading: An ensemble strategy," in *Proceedings of the first ACM international conference on AI in finance*, 2020, pp. 1–8.

[18] X.-Y. Liu, Z. Xia, H. Yang, J. Gao, D. Zha, M. Zhu, C. D. Wang, Z. Wang, and J. Guo, "Dynamic datasets and market environments for financial reinforcement learning," *Machine Learning*, pp. 1–45, 2024.

[19] Q. Wen, T. Zhou, C. Zhang, W. Chen, Z. Ma, J. Yan, and L. Sun, "Transformers in time series: A survey," *arXiv preprint arXiv:2202.07125*, 2022.

[20] S. Liu, H. Yu, C. Liao, J. Li, W. Lin, A. X. Liu, and S. Dustdar, "Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting," in *International conference on learning representations*, 2021.

[21] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *International Conference on Machine Learning*, PMLR, 2022, pp. 27 268–27 286.

[22] Y. Liu, H. Wu, J. Wang, and M. Long, "Non-stationary transformers: Exploring the stationarity in time series forecasting," in *Advances in Neural Information Processing Systems*, 2022.

[23] R.-G. Cirstea, C. Guo, B. Yang, T. Kieu, X. Dong, and S. Pan, "Triformer: Triangular, variable-specific attentions for long sequence multivariate time series forecasting–full version," *arXiv preprint arXiv:2204.13767*, 2022.

[24] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

[25] A. Das, W. Kong, A. Leach, R. Sen, and R. Yu, "Long-term forecasting with tide: Time-series dense encoder," *arXiv preprint arXiv:2304.08424*, 2023.

[26] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," *arXiv preprint arXiv:2211.14730*, 2022.

[27] Z. Liu, D. Huang, K. Huang, Z. Li, and J. Zhao, "Finbert: A pre-trained financial language representation model for financial text mining," in *Proceedings of the twenty-ninth international conference on international joint conferences on artificial intelligence*, 2021, pp. 4513–4519.

[28] D. Araci, "Finbert: Financial sentiment analysis with pre-trained language models," *arXiv preprint arXiv:1908.10063*, 2019.

[29] Y. Yang, M. C. S. Uy, and A. Huang, "Finbert: A pretrained language model for financial communications," *arXiv preprint arXiv:2006.08097*, 2020.

[30] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[31] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[32] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 30, 2016.

[33] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *International conference on machine learning*, PMLR, 2016, pp. 1995–2003.

[34] L. Chen and Q. Gao, "Application of deep reinforcement learning on automated stock trading," in *2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS)*, IEEE, 2019, pp. 29–33.

[35] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai, "Deep direct reinforcement learning for financial signal representation and trading," *IEEE transactions on neural networks and learning systems*, vol. 28, no. 3, pp. 653–664, 2016.

[36] J. Moody and M. Saffell, "Learning to trade via direct reinforcement," *IEEE transactions on neural Networks*, vol. 12, no. 4, pp. 875–889, 2001.

[37] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[38] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *International conference on machine learning*, PMLR, 2016, pp. 1928–1937.

[39] Z. Zhang, S. Zohren, and S. Roberts, "Deep reinforcement learning for trading," *arXiv preprint arXiv:1911.10107*, 2019.

[40] A. Nan, A. Perumal, and O. R. Zaiane, "Sentiment and knowledge based algorithmic trading with deep reinforcement learning," in *International Conference on Database and Expert Systems Applications*, Springer, 2022, pp. 167–180.

[41] W. Han, B. Zhang, Q. Xie, M. Peng, Y. Lai, and J. Huang, "Select and trade: Towards unified pair trading with hierarchical reinforcement learning," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 4123–4134.

[42] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," *arXiv preprint arXiv:2109.01652*, 2021.

[43] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[44] B. Zhang, H. Yang, T. Zhou, M. Ali Babar, and X.-Y. Liu, "Enhancing financial sentiment analysis via retrieval augmented large language models," in *Proceedings of the Fourth ACM International Conference on AI in Finance*, 2023, pp. 349–356.

[45] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, *et al.*, "Emergent abilities of large language models," *arXiv preprint arXiv:2206.07682*, 2022.

[46] S. Sohangir, D. Wang, A. Pomeranets, and T. M. Khoshgoftaar, "Big data: Deep learning for financial sentiment analysis," *Journal of Big Data*, vol. 5, no. 1, pp. 1–25, 2018.

[47] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[48] OpenAI, *Gpt-4 technical report*, 2023. arXiv: 2303.08774 [cs.CL].

[49] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[50] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, *et al.*, "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022.

[51] Y. Yang, Y. Tang, and K. Y. Tam, "Investlm: A large language model for investment using financial domain instruction tuning," *arXiv preprint arXiv:2309.13064*, 2023.

[52] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," *arXiv preprint arXiv:2106.09685*, 2021.

[53] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *arXiv preprint arXiv:2305.14314*, 2023.

[54] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[55] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[56] S. Wang, H. Yuan, L. M. Ni, and J. Guo, "Quantagent: Seeking holy grail in trading by self-improving large language model," *arXiv preprint arXiv:2402.03755*, 2024.

[57] A. Lopez-Lira and Y. Tang, "Can chatgpt forecast stock price movements? return predictability and large language models," *arXiv preprint arXiv:2304.07619*, 2023.

[58] A. Havrilla, Y. Du, S. C. Raparthy, C. Nalmpantis, J. Dwivedi-Yu, M. Zhuravinskyi, E. Hambro, S. Sukhbaatar, and R. Raileanu, "Teaching large language models to reason with reinforcement learning," *arXiv preprint arXiv:2403.04642*, 2024.

[59] B. Zhang, H. Yang, and X.-Y. Liu, "Instruct-fingpt: Financial sentiment analysis by instruction tuning of general-purpose large language models," *arXiv preprint arXiv:2306.12659*, 2023.

[60] S. Fatemi and Y. Hu, "A comparative analysis of fine-tuned llms and few-shot learning of llms for financial sentiment analysis," *arXiv preprint arXiv:2312.08725*, 2023.

[61] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[62] Anthropic, *Introducing the next generation of claude*, https://www.anthropic.com/news/claude-3-family, Accessed: 2024-03-04, 2024.

[63] G. Team, R. Anil, S. Borgeaud, Y. Wu, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, *et al.*, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[64] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, *et al.*, "A survey on evaluation of large language models," *ACM Transactions on Intelligent Systems and Technology*, 2023.

[65] D. Bertsekas, *Dynamic programming and optimal control: Volume I*. Athena scientific, 2012, vol. 4.

[66] X.-Y. Liu, Z. Xiong, S. Zhong, H. Yang, and A. Walid, "Practical deep reinforcement learning approach for stock trading," *arXiv preprint arXiv:1811.07522*, 2018.

[67] S. Pateria, B. Subagdja, A.-h. Tan, and C. Quek, "Hierarchical reinforcement learning: A comprehensive survey," *ACM Computing Surveys (CSUR)*, vol. 54, no. 5, pp. 1–35, 2021.

[68] M. Qin, S. Sun, W. Zhang, H. Xia, X. Wang, and B. An, "Earnhft: Efficient hierarchical reinforcement learning for high frequency trading," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024, pp. 14 669–14 676.

[69] R. Wang, H. Wei, B. An, Z. Feng, and J. Yao, "Deep stock trading: A hierarchical reinforcement learning framework for portfolio optimization and order execution," *arXiv preprint arXiv:2012.12620*, 2020.

[70] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International conference on machine learning*, PMLR, 2018, pp. 1587–1596.

[71] X. Li, Y. Li, Y. Zhan, and X.-Y. Liu, "Optimistic bull or pessimistic bear: Adaptive deep reinforcement learning for stock portfolio allocation," *arXiv preprint arXiv:1907.01503*, 2019.

[72] P. Koratamaddi, K. Wadhwani, M. Gupta, and S. G. Sanjeevi, "Market sentiment-aware deep reinforcement learning approach for stock portfolio allocation," *Engineering Science and Technology, an International Journal*, vol. 24, no. 4, pp. 848–859, 2021.

[73] A. Millea, "Deep reinforcement learning for trading—a critical survey," *Data*, vol. 6, no. 11, p. 119, 2021.

[74] H. Yang, X.-Y. Liu, and Q. Wu, "A practical machine learning approach for dynamic stock recommendation," in *2018 17th IEEE international conference on trust, security and privacy in computing and communications/12th IEEE international conference on big data science and engineering (TrustCom/BigDataSE)*, IEEE, 2018, pp. 1693–1697.

[75] X.-Y. Liu, H. Yang, J. Gao, and C. D. Wang, "Finrl: Deep reinforcement learning framework to automate trading in quantitative finance," in *Proceedings of the second ACM international conference on AI in finance*, 2021, pp. 1–9.

[76] M. Sheth, A. Gerovitch, R. Welsch, and N. Markuzon, "The univariate flagging algorithm (ufa): An interpretable approach for predictive modeling," *PloS one*, vol. 14, no. 10, e0223161, 2019.

[77] K. O'shea and R. Nash, "An introduction to convolutional neural networks," *arXiv preprint arXiv:1511.08458*, 2015.

[78] J. H. Friedman, "Multivariate adaptive regression splines," *The annals of statistics*, vol. 19, no. 1, pp. 1–67, 1991.

[79] M. T. Ribeiro, S. Singh, and C. Guestrin, """ why should i trust you?""" explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 2016, pp. 1135–1144.

[80] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in neural information processing systems*, vol. 30, 2017.

[81] D. Donoho and J. Jin, "Higher criticism thresholding: Optimal feature selection when useful features are rare and weak," *Proceedings of the National Academy of Sciences*, vol. 105, no. 39, pp. 14 790–14 795, 2008.

[82] H. Lakkaraju, S. H. Bach, and J. Leskovec, "Interpretable decision sets: A joint framework for description and prediction," in *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 2016, pp. 1675–1684.

[83] B. A. Williams, J. Mandrekar, S. J. Mandrekar, S. S. Cha, and A. Furth, "Finding optimal cutpoints for continuous covariates with binary and time-to-event outcomes," 2006.

[84] T. Hastie and R. Tibshirani, "Generalized additive models: Some applications," *Journal of the American Statistical Association*, vol. 82, no. 398, pp. 371–386, 1987.

[85] F. Wang, X. Shu, I. Meszoely, T. Pal, I. A. Mayer, Z. Yu, W. Zheng, C. E. Bailey, and X.-O. Shu, "Overall mortality after diagnosis of breast cancer in men vs women," *JAMA oncology*, vol. 5, no. 11, pp. 1589–1596, 2019.

[86] A. Tchagna Kouanou, T. Mih Attia, C. Feudjio, A. F. Djeumo, A. Ngo Mouelas, M. P. Nzogang, C. Tchito Tchapga, and D. Tchiotsop, "An overview of supervised machine learning methods and data analysis for covid-19 detection," *Journal of Healthcare Engineering*, vol. 2021, 2021.

[87] K. P. Bennett and O. L. Mangasarian, "Robust linear programming discrimination of two linearly inseparable sets," *Optimization methods and software*, vol. 1, no. 1, pp. 23–34, 1992.

[88] R. F. Tate, "Correlation between a discrete and a continuous variable. point-biserial correlation," *The Annals of mathematical statistics*, vol. 25, no. 3, pp. 603–607, 1954.

[89] H. Cramér, *Mathematical methods of statistics*. Princeton university press, 1999, vol. 43.

[90] T. R. Fleming and D. P. Harrington, *Counting processes and survival analysis*. John Wiley & Sons, 2011.

[91] J. W. Smith, J. E. Everhart, W. Dickson, W. C. Knowler, and R. S. Johannes, "Using the adap learning algorithm to forecast the onset of diabetes mellitus," in *Proceedings of the annual symposium on computer application in medical care*, American Medical Informatics Association, 1988, p. 261.

[92] T. R. Golub, D. K. Slonim, P. Tamayo, C. Huard, M. Gaasenbeek, J. P. Mesirov, H. Coller, M. L. Loh, J. R. Downing, M. A. Caligiuri, *et al.*, "Molecular classification of cancer: Class discovery and class prediction by gene expression monitoring," *science*, vol. 286, no. 5439, pp. 531–537, 1999.

[93] T. Ryffel, A. Trask, M. Dahl, B. Wagner, J. Mancuso, D. Rueckert, and J. Passerat-Palmbach, "A generic framework for privacy preserving deep learning," *arXiv preprint arXiv:1811.04017*, 2018.

[94] Y. Liu, Q. Liu, H. Zhao, Z. Pan, and C. Liu, "Adaptive quantitative trading: An imitative deep reinforcement learning approach," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, 2020, pp. 2128–2135.